

**IMPLEMENTATION OF DAG AND CPM IN THE ASSEMBLY
SCHEDULING OF THE HONDA RA621H FORMULA 1 ENGINE
FINAL SEMESTER EXAM PROJECT REPORT**

(Prepared to fulfill the requirements for passing the Final Exam Semester 2)



By :

Ababil Khoerul Imam (25031554152)

Ghita Natasha Putri (25031554023)

Orchidea Savira Putri Arief Dhidin (25031554174)

Lecturer:

Yuni Rosita Dewi, S.Si., M.Si.

Kartika Chandra Dewi, S.Si., M.Si.

**STUDY PROGRAM DATA SCIENCE
FACULTY OF MATHEMATICS AND NATURAL SCIENCES
STATE UNIVERSITY OF SURABAYA
2025**

TABLE OF CONTENTS

JUDUL	1
TABLE OF CONTENTS.....	1
CHAPTER 1 INTRODUCTION	2
1.1 BACKGROUND.....	2
1.2 REFERENCE JUSTIFICATION (MINIMUM SINTA 4).....	3
CHAPTER 2 PROJECT OBJECTIVES.....	4
CHAPTER 3 DATA STRUCTURES USED	6
CHAPTER 4 EXPECTED OUTPUT	10
CHAPTER 5 ALGORITHM AND WORKFLOW	20
CHAPTER 6 ATTACHMENT.....	48
REFERENCES.....	49

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Dependency-based project scheduling is one of the main challenges in complex engineering systems. In manufacturing and industrial environments, mistakes in arranging task order can affect the entire production process, not just waste time. Kumari and Singh (2024) explained in the *International Research Journal of Management Science & Technology* that **CPM (Critical Path Method)** and PERT (Program Evaluation and Review Technique) are important scheduling methods that use the concept of **Directed Acyclic Graph (DAG)**, where nodes represent tasks and edges represent dependency relationships between tasks.

This problem becomes more important when the production scale gets larger. Liu and Lv (2023), in *Intelligent Automation & Soft Computing*, showed that in garment assembly lines with a small-batch multi-variety (SBMV) production system, using **BFS-based topological sorting** can arrange assembly tasks more efficiently compared to manual methods. Their research showed an increase of more than 15% in assembly line efficiency. The study also proved that processes with a DAG structure can be solved more effectively by combining topological sort with optimization methods.

From a theoretical perspective, Vazquez et al. (2023) in *Scientific Reports (Nature)* explained that the structure of an activity network greatly affects overall project performance. Based on data from 77 real-world construction projects with a total budget of more than \$10 billion, they found that as network complexity increases, the percentage of activities included in the critical path becomes smaller. This means that managing the whole dependency network is more important than only focusing on the critical path.

This project applies those concepts to a real-world case study: the assembly scheduling of the Honda RA621H, a 1.6L V6 turbo-hybrid power unit used by Max Verstappen during the 2021 Formula 1 World Championship season. The dataset contains 103 components divided into 19 subsystems with 216 dependency

relationships. Without a clear algorithmic approach, arranging the correct assembly order manually would be very difficult.

This project implements three main Data Structures and Algorithms concepts:

- Directed Acyclic Graph (DAG) to represent the dependency relationships between components.
- Kahn's Algorithm (BFS-based Topological Sort) to generate a valid assembly order while also detecting cycles in the graph.
- Critical Path Method (CPM) to calculate ES, EF, LS, LF, and Float values and determine the critical path in the assembly process.

All system modeling, graph construction, dependency analysis, and algorithm implementation processes were designed and developed using Visual Studio Code, with the support of the Pandas and NetworkX libraries to manage component datasets, build dependency graphs, and execute graph-based computations efficiently.

1.2 REFERENCE JUSTIFICATION (MINIMUM SINTA 4)

- **Kumari, M., & Singh, G.P. (2024).** *The Application of Graph Theory to Solving CPM and PERT Problems in Java*. International Research Journal of Management Science & Technology (IRJMST), 15(8), 20–28 [ISSN 2250-1959 / **terindeks Google Scholar - SINTA 4**]. This study explains how graph theory concepts such as DAG, topological sorting, CPM, and PERT can be implemented in programming and project scheduling systems (The main algorithmic systems used in this project).
- **Liu, J., & Lv, Y. (2023).** *A Multi-Object Genetic Algorithm for the Assembly Line Balance Optimization in Garment Flexible Job Shop Scheduling*. Intelligent Automation & Soft Computing (IASC), 37(2), 2421–2439 [**Scopus Q2**]. This research shows that topological sorting can improve efficiency in real assembly-line scheduling problems, especially in complex manufacturing environments.
- **Vazquez, A., Pozzana, I., Kalogridis, G., & Ellinas, C. (2023).** Activity networks determine project performance. Scientific Reports, 13, 509 [**Nature Portfolio / Scopus Q1**]. This paper explains that the structure of a project network plays an important role in determining overall project performance, not only the critical path itself.

CHAPTER 2

PROJECT OBJECTIVES

The project focuses on dependency aware scheduling problems and shows how graph algorithms and topological sorting can be applied in a real world engineering case. According to Kumari & Singh (2024), graph theory in CPM is mainly used for identifying the critical path, managing project schedules, and improving resource allocation. Based on those ideas, the objective of this project are :

2.1. To represent the Honda RA621H assembly process as a Directed Acyclic Graph (DAG)

Containing 103 nodes and 216 dependency connections, while making sure the graph has no cycles so a valid assembly sequence can be generated. This approach is similar to the method used by Liu & Lv (2023) in manufacturing scheduling research.

2.2. To implement Kahn's BFS-based Topological Sort Algorithm manually in Python

In order to determine the correct assembly order for all components. The algorithm is implemented without relying on built-in sorting functions so the process can be understood more clearly step by step.

2.3. To apply the Critical Path Method (CPM)

By calculating ES, EF, LS, LF, and Float values for all 103 components, as well as identifying the components included in the critical path of the assembly process.

2.4. To build an interactive browser-based visualization using HTML, CSS, and JavaScript

That allows users to follow the execution of Kahn's Algorithm step by step, including in-degree updates, wave processing, pseudocode visualization, and the formation of the final topological order directly in the browser using Pyodide.

2.5. To compare the CPM results with the original engineering dataset in order

To understand the difference between mathematically critical components and components considered operationally important in real assembly conditions.

2.6. To show that Data Structures and Algorithms concepts such as DAG, topological sorting, and CPM

Are not only theoretical concepts, but can also be applied directly to real-world engineering and assembly scheduling problems.

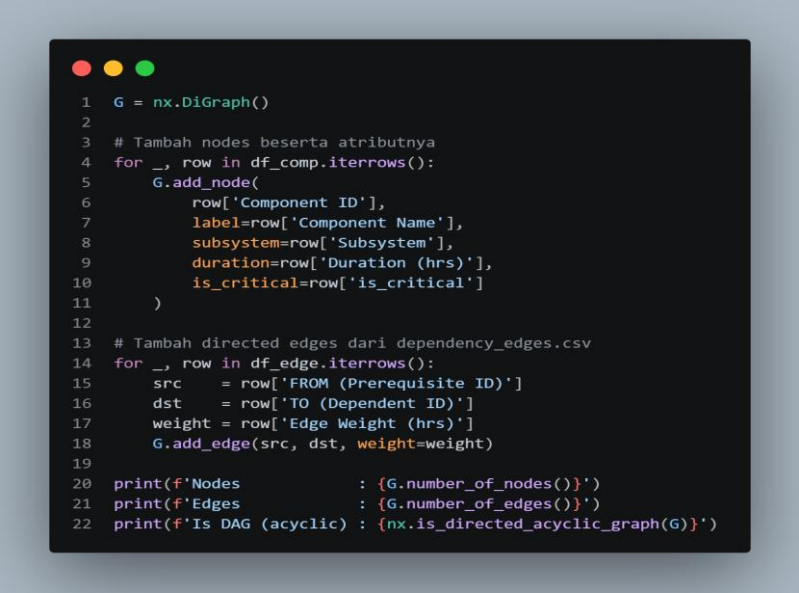
CHAPTER 3

DATA STRUCTURES USED

Liu & Lv (2023) explain that a garment process chart has the same structure as a Directed Acyclic Graph (DAG), where each vertex represents a process and the dependency order must always be followed. The same concept is applied in the Honda RA621H assembly process. Kumari & Singh (2024) also explain that in CPM, nodes represent tasks while edges represent dependency relationships and durations. Based on these ideas, this project uses five main data structures:

A. Directed Acyclic Graph (DAG) — NetworkX DiGraph

A DAG is used because it can clearly represent dependency relationships between assembly components. Vazquez et al. (2023) also explain that real project networks usually have similar in-degree and out-degree distributions. The same pattern appears in this dataset, where the average in-degree and out-degree are both 2.10. In this project, each component becomes a node and each dependency becomes a directed edge. The dataset contains 103 nodes and 216 edges, and the graph is validated to ensure that no cycles exist.



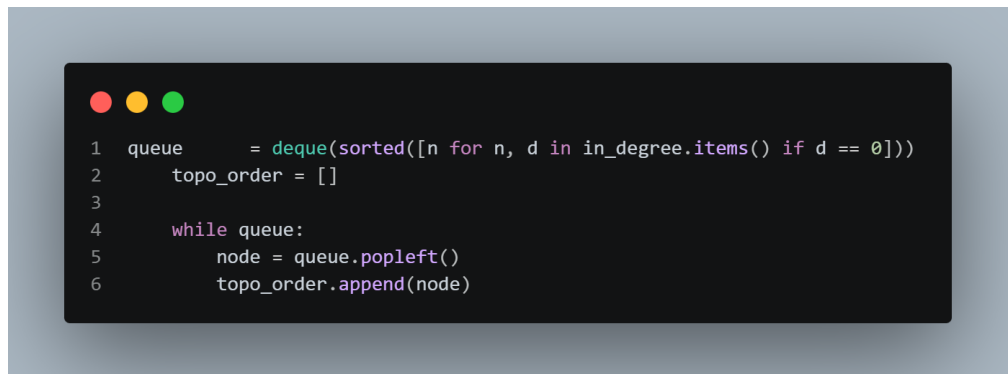
```
1 G = nx.DiGraph()
2
3 # Tambah nodes beserta atributnya
4 for _, row in df_comp.iterrows():
5     G.add_node(
6         row['Component ID'],
7         label=row['Component Name'],
8         subsystem=row['Subsystem'],
9         duration=row['Duration (hrs)'],
10        is_critical=row['is_critical']
11    )
12
13 # Tambah directed edges dari dependency_edges.csv
14 for _, row in df_edge.iterrows():
15     src = row['FROM (Prerequisite ID)']
16     dst = row['TO (Dependent ID)']
17     weight = row['Edge Weight (hrs)']
18     G.add_edge(src, dst, weight=weight)
19
20 print(f'Nodes : {G.number_of_nodes()}')
21 print(f'Edges : {G.number_of_edges()}')
22 print(f'Is DAG (acyclic) : {nx.is_directed_acyclic_graph(G)}')
```

Gambar 3.1 Implementation of DAG in the Algorithm

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

B. Queue (deque) — BFS for Kahn's Algorithm

Kumari & Singh (2024) explain that topological sorting is important for arranging tasks based on their dependencies. Because Kahn's Algorithm uses a BFS approach, a queue data structure is needed. Python's `collections.deque` is used because both `append()` and `popleft()` operations run in $O(1)$ time, keeping the algorithm efficient with $O(V+E)$ complexity. Liu & Lv (2023) also use the same idea by starting from nodes with $\text{in-degree} = 0$ before traversing adjacent nodes.



```
1 queue = deque(sorted([n for n, d in in_degree.items() if d == 0]))
2 topo_order = []
3
4 while queue:
5     node = queue.popleft()
6     topo_order.append(node)
```

Gambar 3.2 A BFS-Based Kahn's Algorithm Using Queue (deque)

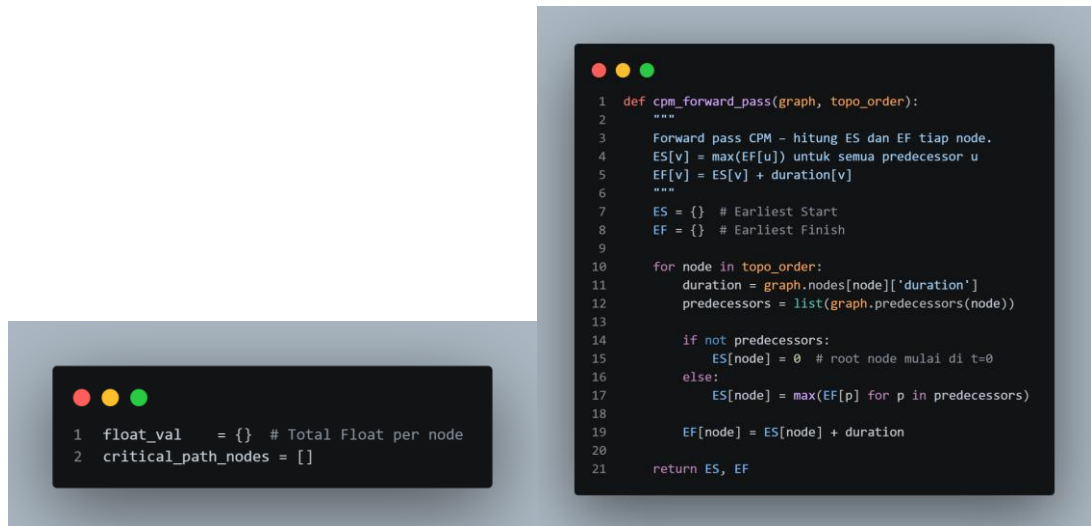
Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

C. Dictionary (defaultdict) — In-Degree Map & CPM Values

Dictionary is used because it allows fast access and updates for every node. Kumari & Singh (2024) state that computing in-degree values is one of the first important steps in Kahn's Algorithm. This structure is also used to store CPM calculations such as ES, EF, LS, LF, and Float values for all components.



```
1 in_degree = defaultdict(int)
2
3 # Hitung in-degree tiap node
4 for node in graph.nodes():
5     if node not in in_degree:
6         in_degree[node] = 0
7     for _ in graph.predecessors(node):
8         in_degree[node] += 1
```

Gambar 3.3 Dictionary (defaultdict) — In-Degree Map & CPM Values

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

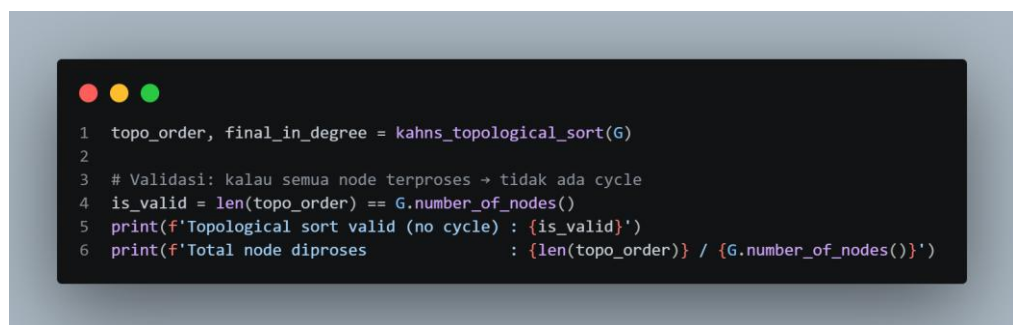
```

in_degree = defaultdict(int)
ES = {n: 0 for n in G.nodes()} # Earliest Start
EF = {n: 0 for n in G.nodes()} # Earliest Finish
float_val = {n: LS[n]-ES[n] for n in G.nodes()} # Total Float

```

D. List — Topological Order & Wave Groups

Liu & Lv (2023) describe the process ordering result as a list structure. In this project, list is used to store the final topological order generated by Kahn's Algorithm. It is also used to store wave groups for the interactive visualization, where nodes in the same wave can be processed in parallel.



Gambar 3.4 Final Topological Order Generated by Kahn's Algorithm

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

```

topo_order = [] # 103 nodes in valid topological order
waves = [] # waves[i] = components workable in parallel at wave i

```

E. DataFrame (pandas) — Dataset Management

The project dataset is stored in Excel format, so pandas DataFrame is used to manage and analyze the data efficiently. Vazquez et al. (2023) also explain that analyzing activity distributions is important for understanding project network characteristics. DataFrame makes it easier to group components by subsystem and calculate assembly statistics.



```
1 df_comp = pd.read_csv('output_csv/components.csv')
2 df_edge = pd.read_csv('output_csv/dependency_edges.csv')
3 df_summary = pd.read_csv('output_csv/summary_stats.csv')
4
5 # Bersihkan & rename kolom komponen
6 df_comp = df_comp[['Component ID', 'Component Name', 'Subsystem',
7                   'Depends On (IDs)', 'Duration (hrs)', 'Is Critical Path']].copy()
8 df_comp['is_critical'] = df_comp['Is Critical Path'].str.contains('CRITICAL', na=False)
9
10 # Bersihkan kolom edges
11 df_edge = df_edge[['FROM (Prerequisite ID)', 'FROM Name',
12                  'TO (Dependent ID)', 'TO Name', 'Edge Weight (hrs)']].copy()
13
14 print(f'Komponen (nodes) : {len(df_comp)}')
15 print(f'Edges           : {len(df_edge)}')
16 df_comp.head()
```

Gambar 3.5 Dataset Management using DataFrame (Pandas)

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

CHAPTER 4

EXPECTED OUTPUT

Following the objectives explained by Kumari & Singh (2024) regarding critical path identification, project scheduling, and resource allocation, this project produces four main outputs :

A. Valid Assembly Sequence (Topological Order)

The first result is a valid assembly order for all 103 components, where every component is placed only after all of its prerequisites are completed. Liu & Lv (2023) explain that topological sorting is used to arrange processes without breaking dependency constraints, and the same principle is applied in this project.

No.	ID	Nama Komponen	Subsystem	Durasi	Kritis?
1	C001	V6 Cylinder Block (1.6L 90° Bank)	ICE – Internal Combustion Engine	48 hrs	★ CRITICAL
2	C018	Compressor Housing	Turbocharger System	18 hrs	★ CRITICAL
3	C025	MGU-H Rotor & Magnet Assembly	Hybrid – MGU-H	20 hrs	★ CRITICAL
4	C030	MGU-K Rotor & Magnet Assembly	Hybrid – MGU-K	18 hrs	★ CRITICAL
5	C035	Li-Ion Battery Cells (ES)	Energy Store (ES)	20 hrs	★ CRITICAL
6	C039	Dry-Sump Oil Tank	Lubrication System	6 hrs	
7	C043	Water Pump (Electric)	Cooling System	6 hrs	
8	C046	Fuel Tank (FIA Bladder, ~100kg capacity)	Fuel System	12 hrs	
9	C049	Engine Control Unit (ECU) – FIA Standard	Electronics & Control	8 hrs	★ CRITICAL
10	C052	Carbon Fibre Monocoque (Survival Cell)	Chassis & Monocoque	96 hrs	★ CRITICAL
11	C002	Crankshaft	ICE – Internal Combustion Engine	24 hrs	★ CRITICAL
12	C019	Compressor Wheel	Turbocharger System	14 hrs	★ CRITICAL
13	C026	MGU-H Stator Windings	Hybrid – MGU-H	16 hrs	★ CRITICAL
14	C031	MGU-K Stator Windings	Hybrid – MGU-K	14 hrs	★ CRITICAL
15	C036	Battery Management System (BMS)	Energy Store (ES)	12 hrs	★ CRITICAL
16	C040	Oil Pump & Scavenge Pumps	Lubrication System	8 hrs	★ CRITICAL
17	C044	Radiators (x2 side pods)	Cooling System	10 hrs	
18	C047	Fuel Pump & Pressure Regulator	Fuel System	6 hrs	★ CRITICAL
19	C050	Wiring Loom (PU section)	Electronics & Control	16 hrs	
20	C053	Roll Hoop (Halo + Main)	Chassis & Monocoque	24 hrs	★ CRITICAL
21	C054	Ballast Weights	Chassis & Monocoque	4 hrs	
22	C056	Front Upper Wishbone (x2)	Suspension – Front	12 hrs	
23	C057	Front Lower Wishbone (x2)	Suspension – Front	12 hrs	
...					
100	C100	Aero & Bodywork Installation	Final Assembly	20 hrs	★ CRITICAL
101	C101	Brakes, Wheels & Tyre Fitting	Final Assembly	8 hrs	★ CRITICAL
102	C102	Cockpit, Safety & Electronics Commissioning	Final Assembly	10 hrs	★ CRITICAL
103	C103	Full Car Systems Check & Sign-Off	Final Assembly	8 hrs	★ CRITICAL

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Gambar 4.1 Table of Valid Assembly Sequence (Topological Order)

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Validation Result :

```
Topological sort valid (no cycle) : True
Total node diproses           : 103 / 103
```

```

RINGKASAN - Honda RA621H DAG + Kahn's Topological Sort
Total nodes          : 103
Total edges          : 216
Is acyclic (valid DAG) : True
Root nodes           : 10
Leaf nodes            : 8
Critical path nodes   : 52
Avg in-degree         : 2.10
Avg out-degree        : 2.10
Kahn's sort valid     : True
Urutan pertama assembly : C001 - V6 Cylinder Block (1.6L 90° Bank)
Urutan terakhir assembly : C103 - Full Car Systems Check & Sign-Off

DAG + Topological Sort selesai. Lanjut ke CPM.

```

Gambar 4.2 Table of Validation result

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipyb (Personal documentation)

B. Full CPM Table (ES, EF, LS, LF, Float per Component)

The second result is a complete CPM table containing ES, EF, LS, LF, and Float values for all 103 components. Vazquez et al. (2023) describe total float as the amount of delay allowed without affecting the final project completion time. In this project, Float is calculated using:

$$\text{Float} = \text{LS} - \text{ES}$$

No.	ID	Nama Komponen	Subsystem	ES	EF	LS	LF	Float	Kritis?
1	C001	V6 Cylinder Block (1.6L 90° Bank)	Internal Combustion Engi	0	48	0	48	0	★ CP
2	C018	Compressor Housing	Turbocharger System	0	18	190	208	190	
3	C025	MGU-H Rotor & Magnet Assembly	MGU-H	0	20	194	214	194	
4	C030	MGU-K Rotor & Magnet Assembly	MGU-K	0	18	212	230	212	
5	C035	Li-Ion Battery Cells (ES)	Energy Store (ES)	0	20	220	240	220	
6	C039	Dry-Sump Oil Tank	Lubrication System	0	6	338	344	338	
7	C043	Water Pump (Electric)	Cooling System	0	6	336	342	336	
8	C046	Fuel Tank (FIA Bladder, ~100kg capacity)	Fuel System	0	12	336	348	336	
9	C049	Engine Control Unit (ECU) - FIA Standard	Electronics & Control	0	8	306	314	306	
10	C052	Carbon Fibre Monocoque (Survival Cell)	Chassis & Monocoque	0	96	104	200	104	
11	C002	Crankshaft	Internal Combustion Engi	48	72	48	72	0	★ CP
12	C019	Compressor Wheel	Turbocharger System	18	32	208	222	190	
13	C026	MGU-H Stator Windings	MGU-H	20	36	214	230	194	
14	C031	MGU-K Stator Windings	MGU-K	18	32	230	244	212	
15	C036	Battery Management System (BMS)	Energy Store (ES)	20	32	240	252	220	
16	C040	Oil Pump & Scavenge Pumps	Lubrication System	6	14	344	352	338	
17	C044	Radiators (x2 side pods)	Cooling System	6	16	342	352	336	
18	C047	Fuel Pump & Pressure Regulator	Fuel System	12	18	348	354	336	
19	C050	Wiring Loom (PU section)	Electronics & Control	8	24	314	330	306	
20	C053	Roll Hoop (Halo + Main)	Chassis & Monocoque	96	120	334	358	238	
21	C054	Ballast Weights	Chassis & Monocoque	96	100	354	358	258	
22	C056	Front Upper Wishbone (x2)	Front	96	108	282	294	186	
23	C057	Front Lower Wishbone (x2)	Front	96	108	282	294	186	
...									
100	C100	Aero & Bodywork Installation	Final Assembly	330	350	330	350	0	★ CP
101	C101	Brakes, Wheels & Tyre Fitting	Final Assembly	330	338	342	350	12	
102	C102	Cockpit, Safety & Electronics Commissioni	Final Assembly	330	340	340	350	10	
103	C103	Full Car Systems Check & Sign-Off	Final Assembly	350	358	350	358	0	★ CP

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Gambar 4.3 Table of Full CPM (ES, EF, LS, LF, Float per Component)

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipyb (Personal documentation)

Project Summary :

Project Duration: 358 hours $\approx$ 45 working days @ 8 hrs/day

Critical nodes: 19

Max float: 338 hrs (C039)

```
Project Duration (Critical Path Length) : 358 jam
Earliest Finish node terakhir (C103)   : 358 jam

Backward pass selesai.
LS node pertama (harus 0) : 0
LF node terakhir          : 358 jam

Jumlah node di Critical Path : 19
Project Duration            : 358 jam
```

Gambar 4.4 Summary

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipyb (Personal documentation)

C. Critical Path (19 Components with Float = 0)

Vazquez et al. (2023) explain that the critical path is the sequence of activities that has no slack time. In this project, 19 components are identified as critical-path nodes because they have Float = 0.

ID	Nama Komponen	ES	EF	LS	LF	Float	Status
C001	V6 Cylinder Block (1.6L 90° Bank)	0	48	0	48	0	★ CRITICAL
C002	Crankshaft	48	72	48	72	0	★ CRITICAL
C003	Connecting Rods (x6)	72	84	72	84	0	★ CRITICAL
C004	Pistons (x6)	84	100	84	100	0	★ CRITICAL
C006	Cylinder Head (x2 banks)	100	136	100	136	0	★ CRITICAL
C007	Intake Valves (x12)	136	144	136	144	0	★ CRITICAL
C008	Exhaust Valves (x12)	136	144	136	144	0	★ CRITICAL
C009	Camshafts (x4)	144	158	144	158	0	★ CRITICAL
C014	Exhaust Manifold (equal-length runners)	144	164	144	164	0	★ CRITICAL
C010	Valve Actuation - Pneumatic System	158	164	158	164	0	★ CRITICAL
C015	Engine Assembly & Bench Calibration	164	200	164	200	0	★ CRITICAL
C055	Mounting Points - Engine & Gearbox	200	208	200	208	0	★ CRITICAL
C077	Gearbox Casing (8-speed semi-auto)	208	256	208	256	0	★ CRITICAL
C078	Gear Stack & Selector Mechanism	256	288	256	288	0	★ CRITICAL
C079	Differential	288	302	288	302	0	★ CRITICAL
C098	Gearbox Mating to PU	302	314	302	314	0	★ CRITICAL
C099	Suspension Assembly - Full Car	314	330	314	330	0	★ CRITICAL
C100	Aero & Bodywork Installation	330	350	330	350	0	★ CRITICAL
C103	Full Car Systems Check & Sign-Off	350	358	350	358	0	★ CRITICAL

Gambar 4.5

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipyb (Personal documentation)

The critical path spans several subsystems :

- ICE (10 nodes)
- Chassis Mounting (1)
- Gearbox (3)
- Final Assembly (4)

An important finding that matches Vazquez et al. (2023) is that only 19 out of 103 components (18.4%) are part of the critical path. This

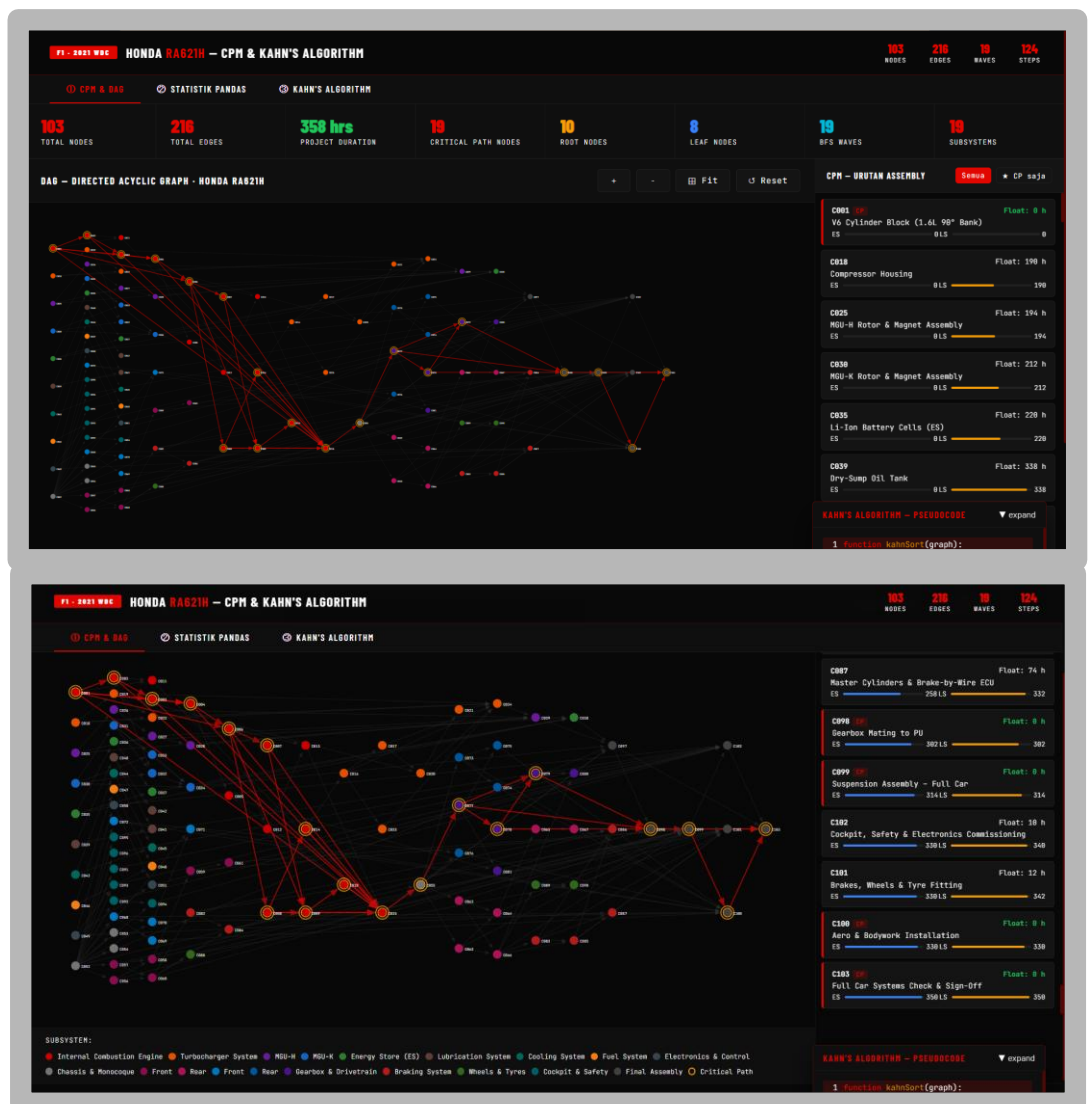
supports the idea that as the network becomes larger, the percentage of critical-path activities becomes smaller.

D. Interactive Web Visualization (index.html)

The final output is an interactive browser-based visualization that runs Kahn's Algorithm using Pyodide (Python running through WebAssembly). The purpose of this visualization is to make the algorithm easier to understand by showing every step directly in the browser.

Main Features:

1. Tab 1 CPM & DAG (Interactive SVG Graph)



Gambar 4.6

Source : Website https://orchydhea007-cell.github.io/honda-ra621h-cpm_revised-last/

(Personal documentation)

Tab 1 is the main visualization tab and serves as the primary interface for exploring the Honda RA621H assembly graph and its Critical Path Method results. The core element of this tab is a fully interactive SVG-based DAG rendered directly inside the browser without any external graph library. All 103 components are drawn as nodes, and all 216 dependency relationships are drawn as directed edges connecting them. The layout is computed using a layered topological ranking algorithm so each node is assigned a rank equal to the length of the longest path from a root node to that node, and nodes with the same rank are placed in the same horizontal column. This ensures that the direction of every edge always flows from left to right, making the assembly dependency order visually clear and easy to follow.

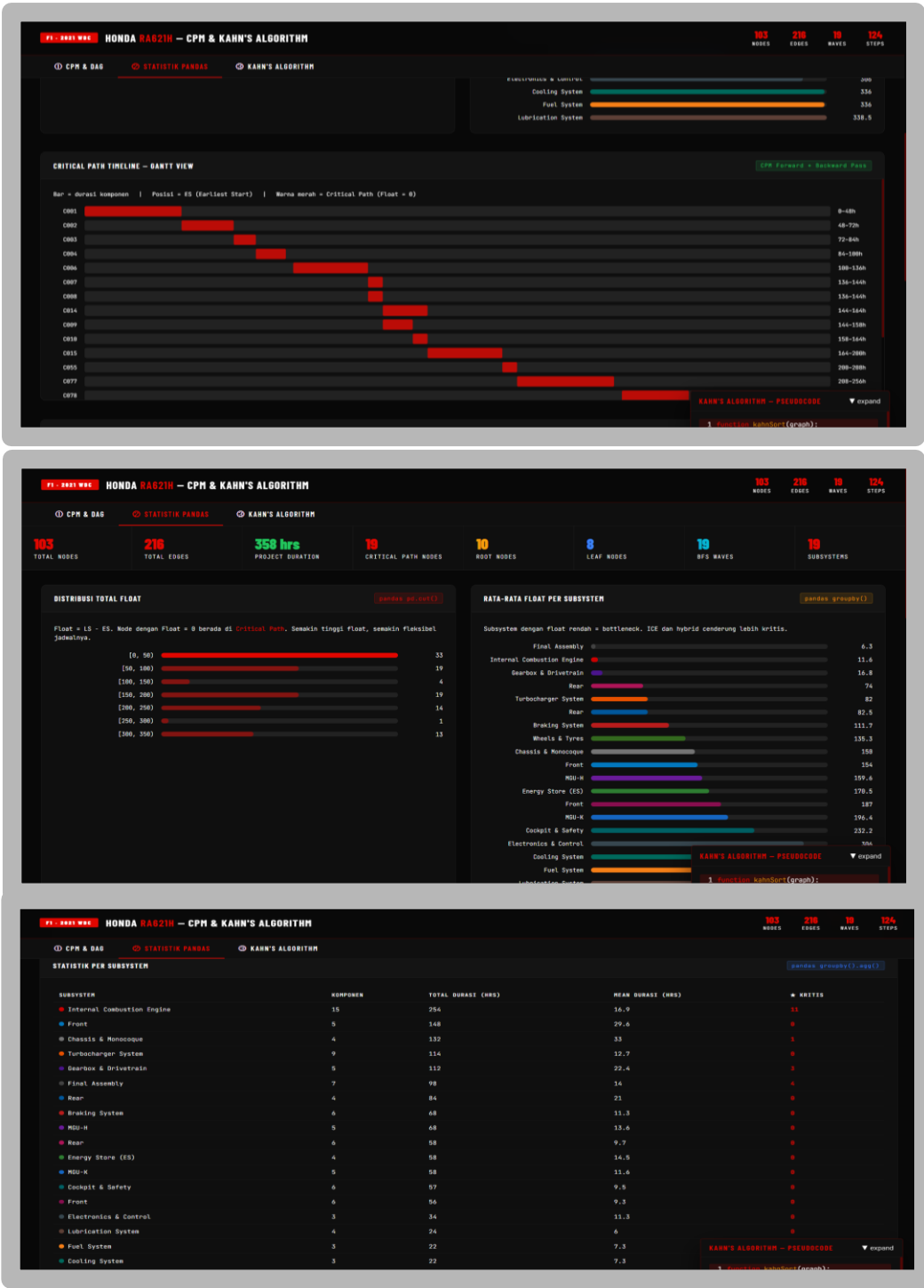
Each node in the SVG is color-coded based on its subsystem using a predefined color palette and totaling 19 distinct subsystem colors. Nodes that belong to the Critical Path are additionally outlined with a bright golden border so that users can immediately distinguish them from non-critical components without needing to consult the CPM table. The SVG canvas supports three interaction modes that is pan (click and drag the canvas to move around the graph), zoom (scroll wheel or dedicated zoom-in/zoom-out buttons to scale the graph up or down), and fit-to-view (a button that automatically resets the transform so all 103 nodes are visible within the viewport). These controls allow users to freely navigate a graph that would otherwise be too large to display in full at a readable scale.

When the user clicks on any node in the SVG, a Node Drawer slides in from the right side of the screen. This drawer displays the full detail of the selected component. The drawer also shows the component's material specification, structural weight, a list of all predecessor IDs (components that must be completed before this one), a list of all successor IDs (components that depend on this one), and any technical notes associated with the component from the dataset. The drawer can be closed either by clicking the close button, clicking the backdrop behind it, or pressing the Escape key on the keyboard.

Below the SVG graph, a CPM List panel displays all 103 nodes arranged in their valid topological order. Each entry in the list shows the node's ID, name, subsystem, duration, and a visual bar representing the proportional positions of its ES (Earliest Start) and LS (Latest Start) values relative to the total project duration

of 358 hours. Nodes on the Critical Path are highlighted in a distinct color within the list. The panel also includes two filter buttons that is “All Nodes” (default, shows all 103 components) and “Critical Path Only” (hides non-critical nodes, showing only the 19 critical-path components for focused review). This combination of the interactive graph and the CPM list allows users to explore both the structural and temporal aspects of the assembly process from a single tab.

2. Tab 2 Pandas Statistics (Data Analysis Dashboard)



Gambar 4.7

Source : Website https://orchydhea007-cell.github.io/honda-ra621h-cpm_revised-last/
(Personal documentation)

Tab 2 is dedicated to the statistical analysis layer of the project, powered entirely by pandas running inside the browser via Pyodide. Unlike static charts generated outside the browser, all computations in this tab are performed at runtime by actual Python code: the dataset is passed from JavaScript into the Pyodide runtime, processed using pandas DataFrame operations, and the results are returned to JavaScript as JSON to be rendered as SVG charts.

The first visualization is the Float Distribution Chart, a vertical bar chart (histogram) generated using `pd.cut()` to divide all 103 components' Float values into discrete bins. Each bar represents the number of components whose Float falls within a specific range. This chart makes it visually clear that the distribution is heavily skewed because the majority of components have either zero float (critical path) or very high float values, with relatively few in the middle ranges.

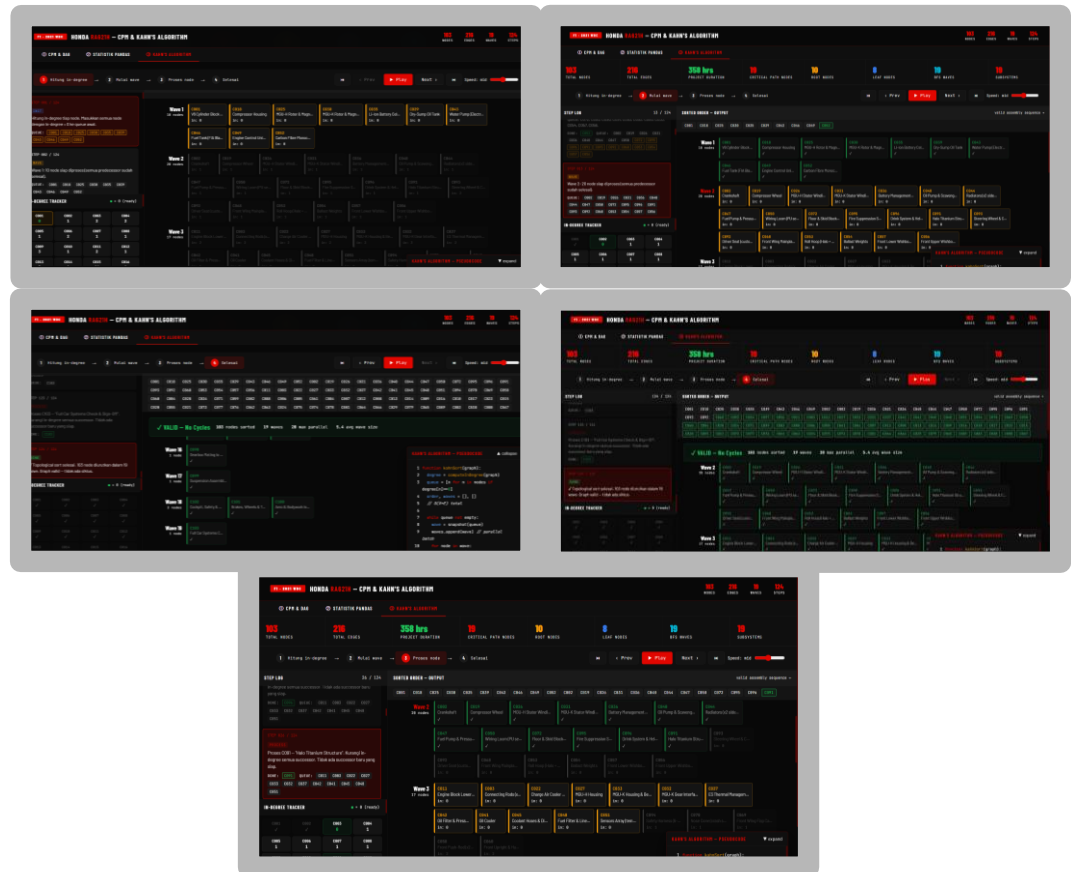
The second visualization is the Average Float per Subsystem Chart, a horizontal bar chart that shows the mean total float of all components grouped by their subsystem. This chart is generated using `groupby('subsystem').agg()` in pandas. Each bar is colored to match that subsystem's color from the DAG tab. Subsystems with a lower average float (closer to the critical path) appear shorter, while subsystems with a higher average float (more scheduling flexibility) appear longer. This allows for a quick comparison of which subsystems are structurally more constrained in the assembly timeline.

The third visualization is the Critical Path Gantt Timeline, which shows only the 19 critical-path components arranged in the order they appear along the critical path. For each component, a horizontal bar is drawn with a lighter segment representing the ES (Earliest Start) offset, and a filled segment representing the duration of the component, positioned proportionally within the total project duration of 358 hours.

The fourth element in this tab is the Subsystem Statistics Table, a structured data table computed via pandas groupby aggregation. Each row in the table represents one of the 19 subsystems and contains five columns that is subsystem

name (with a color dot matching the DAG), the number of components in that subsystem, the total duration of all components combined (in hours), the mean duration per component (in hours, rounded to two decimal places), and the number of critical-path components within that subsystem.

3. Tab 3 Kahn's Algorithm (Step-by-Step Playback)



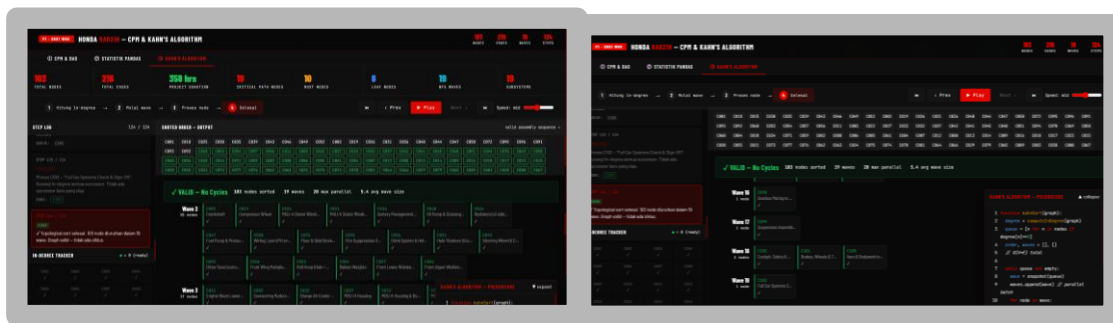
Gambar 4.8

Source : Website https://orchydh007-cell.github.io/honda-ra621h-cpm_revised-last/

(Personal documentation)

Tab 3 is the most educationally focused part of the web visualization. Its purpose is to make Kahn's BFS Topological Sort Algorithm fully transparent and understandable by showing every single step of the algorithm's execution in a controlled, interactive environment. Playback is controlled through a set of navigation buttons at the top of the tab that is Play/Pause (starts or pauses automatic stepping at a configurable speed), Next Step (advances one step forward), Previous Step (goes one step back), First (jumps to the very beginning of the algorithm), and

Last (jumps to the final step where all nodes have been processed).



Gambar 4.9

Source : Website https://orchydeha007-cell.github.io/honda-ra621h-cpm_revised-last/
(Personal documentation)

The Wave Grid is the central visual element of this tab. It organizes all 103 nodes into rows, where each row corresponds to one BFS wave (a group of nodes that can be processed in parallel because they all have in-degree = 0 at the same point in the algorithm). Each node is displayed as a card showing its Component ID, a truncated component name, and its current in-degree value. As the algorithm steps forward, each node card transitions through four visual states that is pending (grey, not yet reachable), queued (highlighted as currently in the BFS queue), active (currently being dequeued and processed), and done (green, already added to the topological order). This color-state system makes it immediately visible which nodes are being processed at any moment and which are still waiting.

The In-Degree Tracker panel is displayed alongside the Wave Grid and shows a grid of all 103 nodes with their current in-degree values. As each step is executed, the in-degree counter of every successor node of the currently processed node is decremented by 1 in real time. Nodes whose in-degree has just reached 0 are visually highlighted as newly eligible for the queue. This panel serves as a direct numerical representation of the state of the algorithm at each moment, making it easy to verify the BFS logic manually while watching the Wave Grid update simultaneously.

The Step Log is a scrollable history panel that records every action taken by the algorithm. Each entry in the log includes a phase badge indicating whether the current action is initialization (loading root nodes into the queue), a dequeue operation (removing a node from the queue and appending it to the order), an in-degree update (decrementing a successor's in-degree), or a validation check

(confirming that the total processed count equals 103, confirming no cycles exist). The log auto-scrolls to keep the latest step visible. At the bottom of the tab, the Order Strip shows the growing sequence of node IDs in the topological order, with each new node appended to the strip as soon as it is processed. Finally, the Pseudocode Overlay displays the pseudocode of Kahn's Algorithm with the currently executing line highlighted in real time, directly linking the visual step to the corresponding line of the algorithm. This feature helps users understand not just what is happening visually, but exactly where in the algorithm's logic each action originates.

CHAPTER 5

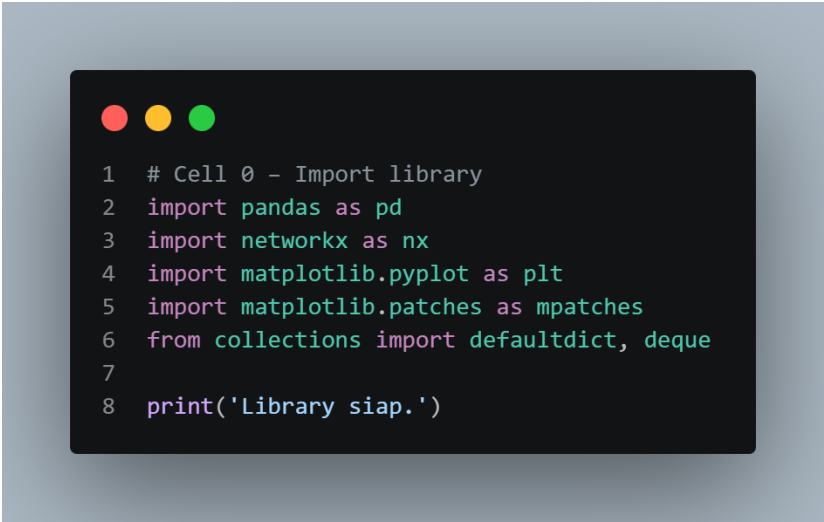
ALGORITHM AND WORKFLOW

Following the objectives explained by Kumari & Singh (2024) regarding critical path identification, project scheduling, and resource allocation, this project produces four main outputs :

A. STAGE 1 - BUILDING THE DAG (CELL 0,1,2,3)

The first step in the notebook is importing all libraries needed for the project. Each library has a specific function that supports the implementation process, from reading the dataset to running the algorithms and generating visualizations.

CELL 0 - IMPORT LIBRARY



```
1 # Cell 0 - Import library
2 import pandas as pd
3 import networkx as nx
4 import matplotlib.pyplot as plt
5 import matplotlib.patches as mpatches
6 from collections import defaultdict, deque
7
8 print('Library siap.')
```

Gambar 5.1 Library

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

- **pandas** is used to read and manage the dataset files such as `components.csv`, `dependency_edges.csv`, and `summary_stats.csv`. Using DataFrames makes the component and dependency data easier to organize and analyze.
- **networkx** is used to build the Directed Acyclic Graph (DAG). This library provides the `DiGraph` structure, which is used to create nodes and edges, calculate graph properties, and verify that the graph does not contain cycles using `is_directed_acyclic_graph()`. According to Kumari & Singh (2024), DAG is a suitable structure for representing CPM networks

because nodes can represent tasks while edges represent dependency relationships.

- **matplotlib.pyplot** and **matplotlib.patches** are used to create visualizations throughout the notebook. These include the in-degree and out-degree distribution charts, the complete DAG visualization, as well as the CPM float distribution and Gantt chart generated in later stages.
- **collections.defaultdict** is used during the implementation of Kahn's Algorithm to store and update in-degree values efficiently. Since every node automatically starts with a default value of zero, there is no need to manually check whether a key already exists.
- **collections.deque** is used as the queue structure in the BFS-based implementation of Kahn's Algorithm. This structure is preferred over a standard Python list because both `append()` and `popleft()` operations run in constant time, maintaining the algorithm's overall complexity of:

$$O(V+E)O(V+E)O(V+E)$$

This is consistent with the explanation by Kahn (1962), which forms the basis of the topological sorting algorithm used in this project.

CELL 1 - LOAD DATASET DARI CSV



```
1 # Cell 1 - Load dataset dari CSV
2 df_comp = pd.read_csv('output_csv/components.csv')
3 df_edge = pd.read_csv('output_csv/dependency_edges.csv')
4 df_summary = pd.read_csv('output_csv/summary_stats.csv')
5
6 # Bersihkan & rename kolom komponen
7 df_comp = df_comp[['Component ID', 'Component Name', 'Subsystem',
8                   'Depends On (IDs)', 'Duration (hrs)', 'Is Critical Path']].copy()
9 df_comp['is_critical'] = df_comp['Is Critical Path'].str.contains('CRITICAL', na=False)
10
11 # Bersihkan kolom edges
12 df_edge = df_edge[['FROM (Prerequisite ID)', 'FROM Name',
13                  'TO (Dependent ID)', 'TO Name', 'Edge Weight (hrs)']].copy()
14
15 print(f'Komponen (nodes) : {len(df_comp)}')
16 print(f'Edges : {len(df_edge)}')
17 df_comp.head()
```

Gambar 5.2 Load Dataset Dari Csv

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Input: components.csv (103 rows) and dependency_edges.csv (216 rows), which are loaded using pandas from the output_csv/ folder.

At this stage, the dataset is first prepared and organized before the graph is created. From components.csv, the columns used are Component ID, Component Name, Subsystem, Depends On (IDs), Duration (hrs), and Is Critical Path. An additional column called is_critical is created by checking whether the word "CRITICAL" appears in the Is Critical Path column.

From dependency_edges.csv, the columns FROM (prerequisite component), TO (dependent component), and Edge Weight (hrs) are used to define the dependency relationships between components.

After the data preparation is complete, a graph is created using nx.DiGraph(). Each component is added as a node with several attributes, including the component name (label), subsystem, duration, and critical-path status (is_critical). Next, every dependency relationship is added as a directed edge from the prerequisite component to the dependent component. The edge weight is set based on the duration of the prerequisite component.

CELL 2 - BANGUN DAG DENGAN NETWORKX

```
1 # Cell 2 - Bangun DAG dengan NetworkX
2 G = nx.DiGraph()
3
4 # Tambah nodes beserta atributnya
5 for _, row in df_comp.iterrows():
6     G.add_node(
7         row['Component ID'],
8         label=row['Component Name'],
9         subsystem=row['Subsystem'],
10        duration=row['Duration (hrs)'],
11        is_critical=row['is_critical']
12    )
13
14 # Tambah directed edges dari dependency_edges.csv
15 for _, row in df_edge.iterrows():
16     src = row['FROM (Prerequisite ID)']
17     dst = row['TO (Dependent ID)']
18     weight = row['Edge Weight (hrs)']
19     G.add_edge(src, dst, weight=weight)
20
21 print(f'Nodes : {G.number_of_nodes()}')
22 print(f'Edges : {G.number_of_edges()}')
23 print(f'Is DAG (acyclic) : {nx.is_directed_acyclic_graph(G)}')
```

Gambar 5.3 Bangun DAG dengan Networkx

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

This implementation follows the concept described by Kumari & Singh (2024), who explain that in a CPM-based DAG, nodes represent activities while edges represent dependency relationships and durations. A similar idea is also discussed by Liu & Lv (2023), who state that production process networks can be represented as Directed Acyclic Graphs where dependency order must always be maintained.

OUTPUT :

```
Nodes          : 103
Edges          : 216
Is DAG (acyclic) : True
```

Gambar 5.4 Output Ccll 2

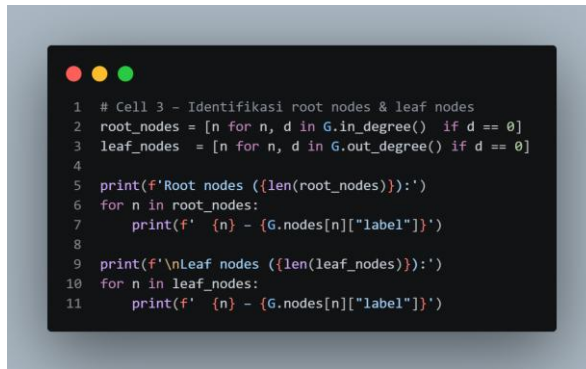
Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The result "Is DAG (acyclic): True" is an important validation because it shows that there are no circular dependencies in the graph. This means a valid assembly sequence can be generated successfully.

In addition, the graph characteristics are consistent with the findings of Vazquez et al. (2023), where real-world activity networks tend to have similar in-degree and out-degree distributions. In this dataset, the average in-degree and out-degree are both equal to 2.10.

B. STAGE 2 - ROOT NODES, LEAF NODES, AND SUBSYSTEM ANALYSIS (CELL 3,4,5,6)

CELL 3 - IDENTIFIKASI ROOT NODES & LEAF NODES



```
1 # Cell 3 - Identifikasi root nodes & leaf nodes
2 root_nodes = [n for n, d in G.in_degree() if d == 0]
3 leaf_nodes = [n for n, d in G.out_degree() if d == 0]
4
5 print(f'Root nodes ({len(root_nodes)}):')
6 for n in root_nodes:
7     print(f' {n} - {G.nodes[n]["label"]}')
8
9 print(f'\nLeaf nodes ({len(leaf_nodes)}):')
10 for n in leaf_nodes:
11     print(f' {n} - {G.nodes[n]["label"]}')
```

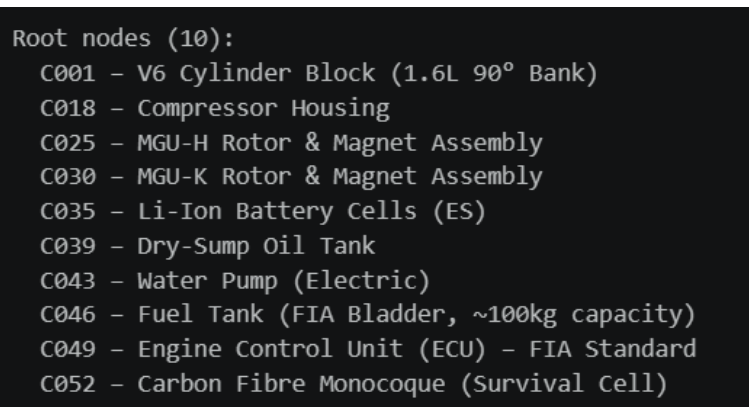
Gambar 5.5 Root & Leaf Nodes

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Before running Kahn's Topological Sort, the graph is first analyzed to understand its overall structure. One of the first steps is identifying the root nodes, which are nodes with:

In-Degree=0

These nodes do not depend on any other components, meaning they can be assembled immediately at the start of the project. The analysis identified 10 root nodes:



```
Root nodes (10):
C001 - V6 Cylinder Block (1.6L 90° Bank)
C018 - Compressor Housing
C025 - MGU-H Rotor & Magnet Assembly
C030 - MGU-K Rotor & Magnet Assembly
C035 - Li-Ion Battery Cells (ES)
C039 - Dry-Sump Oil Tank
C043 - Water Pump (Electric)
C046 - Fuel Tank (FIA Bladder, ~100kg capacity)
C049 - Engine Control Unit (ECU) - FIA Standard
C052 - Carbon Fibre Monocoque (Survival Cell)
```

Gambar 5.6 Output Cell 3 (Root Nodes)

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Since these components have no prerequisites, they can all be started at the same time by different teams during the first stage of assembly.

The analysis also identifies leaf nodes, which are nodes with:

Out-Degree=0

These components do not act as prerequisites for any other component. Among them, C103 – Full Car Systems Check & Sign-Off is the final node that marks the completion of the entire assembly process.

```

Leaf nodes (8):
C041 – Oil Cooler
C042 – Oil Filter & Pressure Regulators
C045 – Coolant Hoses & Distribution Lines
C048 – Fuel Filter & Lines
C053 – Roll Hoop (Halo + Main)
C054 – Ballast Weights
C080 – Driveshafts (x2)
C103 – Full Car Systems Check & Sign-off

```

Gambar 5.7 Output Cell 3 (Leaf Nodes)

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

CELL 4 - STATISTIK DAG PER SUBSYSTEM + SUMMARY STATS

```

1 # Cell 4 - Statistik DAG per subsystem + summary stats
2 subsystem_counts = df_comp.groupby('Subsystem').agg(
3     jumlah_komponen=('Component ID', 'count'),
4     total_durasi=('Duration (hrs)', 'sum'),
5     komponen_kritis=('is_critical', 'sum')
6 ).sort_values('total_durasi', ascending=False)
7
8 print('Statistik per Subsystem:')
9 print(subsystem_counts.to_string())
10
11 # --- Summary Stats dari CSV ---
12 # Format: 2 pasang kolom (metric|value) side by side, baris 0 = header duplikat
13 raw = df_summary.copy()
14 raw.columns = ['metric_1', 'value_1', 'metric_2', 'value_2']
15
16 # Ambil pasangan kiri & kanan, drop baris kosong / header
17 left = raw[['metric_1', 'value_1']].rename(columns={'metric_1': 'metric', 'value_1': 'value'})
18 right = raw[['metric_2', 'value_2']].rename(columns={'metric_2': 'metric', 'value_2': 'value'})
19 stats = pd.concat([left, right]).dropna(subset=['metric'])
20 stats = stats[~stats['metric'].isin(['Metric', 'Subsystem', 'Dataset Summary - Honda RA621H F1 Assembly',
21                                     'Subsystem Breakdown', 'NaN'])]
22 stats = stats[~stats['value'].isin(['Value', 'Color Code', float('nan')])].reset_index(drop=True)
23
24 print('\nDataset Summary - Honda RA621H:')
25 print(f'{"Metric":<40} {"Value":>10}')
26 print('-' * 52)
27 for _, row in stats.iterrows():
28     if str(row['metric']) not in ['nan', 'Total Duration (hrs)':
29         print(f'{str(row["metric"]):<40} {str(row["value"]):>10}')

```

Gambar 5.8 Statistik DAG per Subsystem

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

After identifying the graph structure, a subsystem analysis is performed using pandas groupby(). Three metrics are calculated for each subsystem:

- Number of components
- Total assembly duration (hours)
- Number of critical components

Statistik per Subsystem:			
Subsystem	jumlah_komponen	total_durasi	komponen_kritis
ICE - Internal Combustion Engine	15	254	9
Aerodynamics - Front	5	148	3
Chassis & Monocoque	4	132	3
Turbocharger System	9	114	6
Gearbox & Drivetrain	5	112	4
Final Assembly	7	98	7
Aerodynamics - Rear	4	84	2
Braking System	6	68	2
Hybrid - MGU-H	5	68	4
Hybrid - MGU-K	5	58	4
Energy Store (ES)	4	58	3
Suspension - Rear	6	58	0
Cockpit & Safety	6	57	2
Suspension - Front	6	56	0
Electronics & Control	3	34	1
Lubrication System	4	24	1
Cooling System	3	22	0
Fuel System	3	22	1
Wheels & Tyres	3	18	0

Dataset Summary - Honda RA621H:	
Metric	Value
...	
68	#B71C1C
18	#33691E
57	#006064
98	#212121

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

Gambar 5.9 Output Cell 4

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The results show several interesting findings.

- **The Internal Combustion Engine (ICE) subsystem** contains 15 components with a total duration of 254 hours, including 9 critical components. This makes it the largest and most important subsystem because many other subsystems depend on it directly or indirectly.
- **The Front Aerodynamics subsystem** consists of 5 components with a total duration of 148 hours. Its relatively high duration comes from the complexity of carbon-fiber manufacturing and aerodynamic testing.
- **The Chassis & Monocoque subsystem** contains 4 components with a total duration of 132 hours. This subsystem includes C052 – Carbon Fibre Monocoque, which requires 96 hours and is the single longest activity in the dataset.
- **The Final Assembly subsystem** contains 7 components with a combined duration of 98 hours. All seven components are classified as critical, meaning each step must be completed in sequence without delays.
- Meanwhile, **subsystems such as Wheels & Tyres, Cooling System, and Fuel System** have much shorter durations, ranging from 18 to 22 hours, and contain no critical components. Because of this, they offer greater

scheduling flexibility and can be completed whenever resources are available.

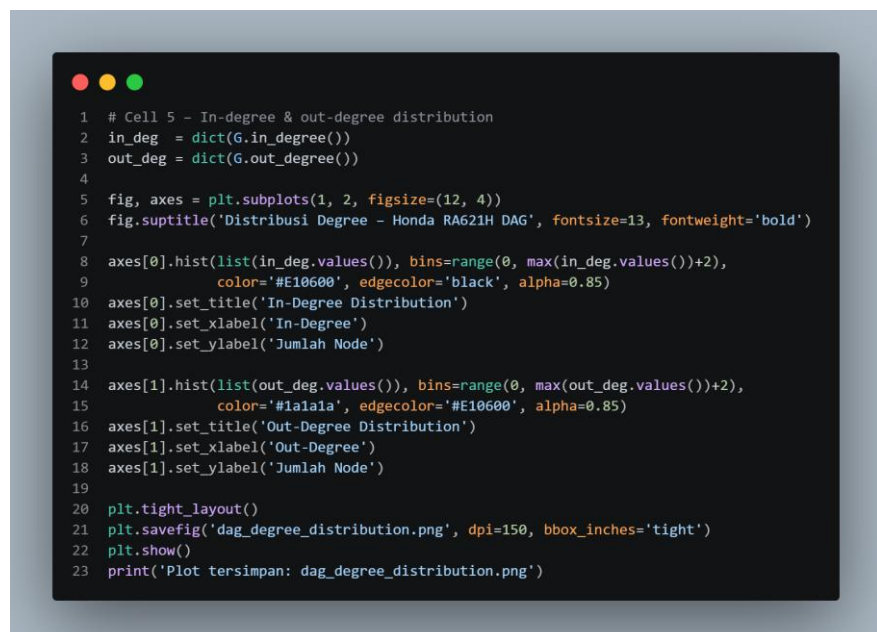
According to the summary generated from summary_stats.csv, the dataset contains:

- 103 components
- 216 dependency edges
- 19 subsystems
- 52 components marked as critical in the dataset
- Total estimated assembly duration of 1,032 hours across all workstreams

These results provide an overview of the assembly network before the topological sorting and CPM calculations are performed. The findings also support the observations of Liu & Lv (2023), who explain that dependency-based production systems can be effectively analyzed through graph structures, while Vazquez et al. (2023) show that understanding network characteristics is important for evaluating overall project performance.

CELL 5 - IN-DEGREE AND OUT-DEGREE DISTRIBUTION ANALYSIS

After the DAG is successfully created, the next step is analyzing the connectivity of each component by calculating its in-degree and out-degree using:

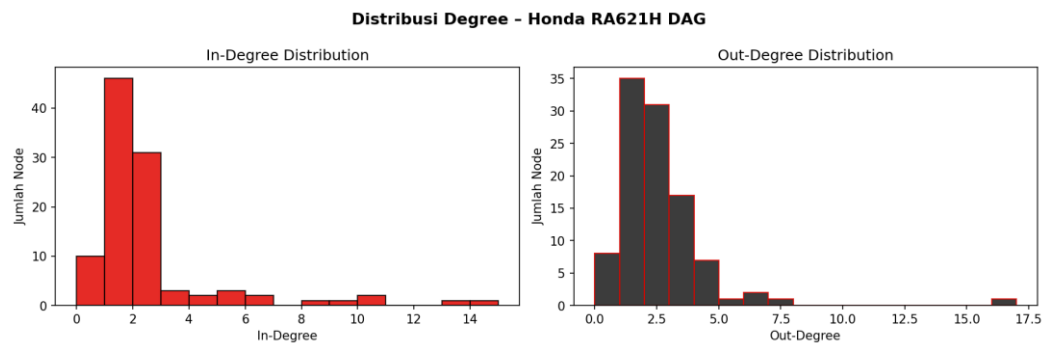


```
1 # Cell 5 - In-degree & out-degree distribution
2 in_deg = dict(G.in_degree())
3 out_deg = dict(G.out_degree())
4
5 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
6 fig.suptitle('Distribusi Degree - Honda RA621H DAG', fontsize=13, fontweight='bold')
7
8 axes[0].hist(list(in_deg.values()), bins=range(0, max(in_deg.values())+2),
9             color='#E10600', edgecolor='black', alpha=0.85)
10 axes[0].set_title('In-Degree Distribution')
11 axes[0].set_xlabel('In-Degree')
12 axes[0].set_ylabel('Jumlah Node')
13
14 axes[1].hist(list(out_deg.values()), bins=range(0, max(out_deg.values())+2),
15             color='#1a1a1a', edgecolor='#E10600', alpha=0.85)
16 axes[1].set_title('Out-Degree Distribution')
17 axes[1].set_xlabel('Out-Degree')
18 axes[1].set_ylabel('Jumlah Node')
19
20 plt.tight_layout()
21 plt.savefig('dag_degree_distribution.png', dpi=150, bbox_inches='tight')
22 plt.show()
23 print('Plot tersimpan: dag_degree_distribution.png')
```

Gambar 5.10 In-Degree And Out-Degree Distribution Analysis

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The results are visualized using two histograms generated with Matplotlib.



Gambar 5.11 Graph In-Degree And Out-Degree Distribution Analysis

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipyb (Personal documentation)

The first histogram shows the **in-degree distribution**, which indicates how many prerequisite components are required before a component can begin. Most components have an in-degree between 1 and 3, meaning they only depend on a small number of preceding components. However, several components have significantly higher in-degree values. For example, **C015 – Engine Assembly & Bench Calibration** can only start after all previous ICE components have been completed, while **C097 – PU Installation into Chassis** must wait for multiple major subsystems to finish before assembly can continue.

The second histogram shows the **out-degree distribution**, which represents how many subsequent components depend on a particular component. Most nodes have an out-degree of only 1 or 2. Components with larger out-degree values play a more important role because delays in these components can affect several downstream activities at the same time. One example is **C001 – V6 Cylinder Block**, which serves as the starting point for much of the ICE assembly chain.

The pattern observed in both histograms is consistent with the findings of Vazquez et al. (2023), who reported that real-world activity networks often exhibit similar in-degree and out-degree distributions. In this project, the average values are:

$$\text{Average In-Degree} = \text{Average Out-Degree} = 103 / 216 \approx 2.10$$

This equality occurs naturally because the graph contains 103 nodes and 216 directed edges, meaning the total number of incoming and outgoing connections must be the same across the network.

CELL 6 — FULL DAG VISUALIZATION USING SPRING LAYOUT

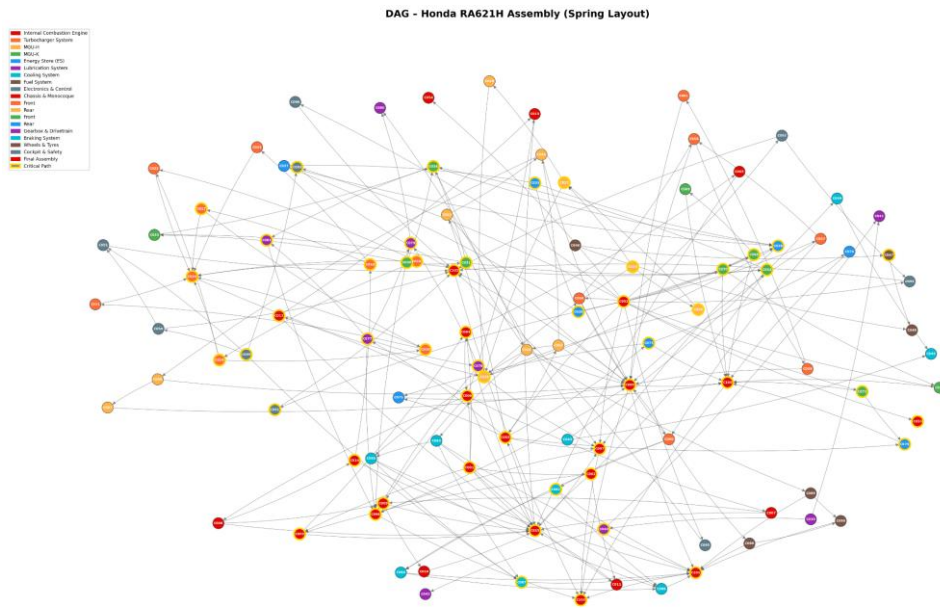
To better understand the overall structure of the assembly network, the complete DAG is visualized using NetworkX's `spring_layout()` algorithm with:

```
1 # Cell 6 - Visualisasi DAG (subsystem view, spring layout)
2
3 # Ambil color code per subsystem dari summary_stats jika tersedia
4 raw2 = df_summary.copy()
5 raw2.columns = ['metric_1', 'value_1', 'metric_2', 'value_2']
6 subsys_block = raw2[raw2['metric_1'] == 'Subsystem'].index
7 if len(subsys_block) > 0:
8     idx_start = subsys_block[0] + 1
9     df_colors = raw2.iloc[idx_start:].dropna(subset=['metric_1', 'value_2'])
10    df_colors = df_colors[df_colors['value_2'].str.startswith('#', na=False)]
11    color_map_csv = dict(zip(df_colors['metric_1'], df_colors['value_2']))
12 else:
13     color_map_csv = {}
14
15 # Fallback palette kalau color tidak ada di CSV
16 fallback_palette = [
17     '#E10600', '#FF6B35', '#FFB347', '#4CAF50',
18     '#2196F3', '#9C27B0', '#00BCD4', '#795548', '#607D8B'
19 ]
20 subsystems = df_comp['Subsystem'].unique()
21 subsystem_color = {}
22 for i, s in enumerate(subsystems):
23     subsystem_color[s] = color_map_csv.get(s, fallback_palette[i % len(fallback_palette)])
24
25 node_colors = [subsystem_color.get(G.nodes[n].get('subsystem', ''), '#999999') for n in G.nodes()]
26 node_edge_colors = ['#FFD700' if G.nodes[n]['is_critical'] else '#333333' for n in G.nodes()]
27 node_linewidths = [2.5 if G.nodes[n]['is_critical'] else 0.5 for n in G.nodes()]
28
29 plt.figure(figsize=(22, 14))
30 plt.title('DAG - Honda RA621H Assembly (Spring Layout)', fontsize=15, fontweight='bold', pad=15)
31
32 pos = nx.spring_layout(G, seed=42, k=1.5)
33
34 nx.draw_networkx_nodes(G, pos, node_color=node_colors, node_size=300,
35                        edgecolors=node_edge_colors, linewidths=node_linewidths)
36 nx.draw_networkx_edges(G, pos, edge_color='#555555', arrows=True,
37                        arrowsize=10, width=0.6, alpha=0.6,
38                        connectionstyle='arc3,rad=0.05')
39 nx.draw_networkx_labels(G, pos, labels={n: n for n in G.nodes()},
40                        font_size=5, font_color='white', font_weight='bold')
41
42 patches = [mpatches.Patch(color=subsystem_color[s], label=s.split('-')[1].strip())
43            for s in subsystems]
44 patches.append(mpatches.Patch(facecolor='gray', edgecolor='#FFD700',
45                               linewidth=2.5, label='Critical Path'))
46 plt.legend(handles=patches, loc='upper left', fontsize=7, framealpha=0.85)
47
48 plt.axis('off')
49 plt.tight_layout()
50 plt.show()
51
```

Gambar 5.12 Full Dag Visualization Using Spring Layout

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Using a fixed seed ensures that the graph layout remains consistent whenever the notebook is executed.



Gambar 5.13 Full Dag Visualization Using Spring Layout

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Each node is colored according to its subsystem using color information obtained from `summary_stats.csv`. Components marked as critical (`is_critical = True`) are highlighted with a gold border (`#FFD700`), while non-critical components use a darker border color.

The visualization reveals several interesting characteristics of the Honda RA621H assembly network.

Subsystem Clustering

The spring layout algorithm places highly connected nodes closer together and pushes loosely connected nodes farther apart. As a result, components belonging to the same subsystem naturally form groups. The ICE, Gearbox, and Final Assembly subsystems appear near the center because they are connected to many other components throughout the network.

Central Hub Components

Several nodes appear near the center of the graph because they connect multiple subsystems together. Examples include:

- C015 – Engine Assembly & Bench Calibration
- C055 – Mounting Points
- C077 – Gearbox Casing
- C097 – PU Installation into Chassis

These components act as important connection points where multiple assembly paths meet before work can continue.

Peripheral Components

Subsystems such as Wheels & Tyres, Cooling System, Fuel System, and Lubrication System are positioned near the edges of the graph. Since they have relatively fewer dependencies, they can often be assembled independently and in parallel with other workstreams.

Critical Components

Components marked as critical are easily recognized by their gold borders. Instead of being randomly distributed, many of these nodes form a visible path through the center of the network, connecting the early ICE components to the final assembly stages.

Edge Directions

Each arrow represents a dependency relationship and points from a prerequisite component to the dependent component. Curved edges are used to improve readability when multiple connections overlap.

Because every arrow moves forward and no edge returns to a previous node, the visualization also provides visual confirmation that the graph is acyclic, supporting the result obtained earlier using `nx.is_directed_acyclic_graph()`.

Overall, this visualization provides a clear overview of the entire Honda RA621H assembly network and helps explain how dependencies are distributed across the 103 components and 216 relationships described in the dataset.

C. STAGE 3 - KAHN'S TOPOLOGICAL SORT (BFS (CELL 7,8,9))

After the DAG was successfully created, Kahn's Algorithm was implemented manually through the `kahns_topological_sort()` function instead of using NetworkX's built-in `topological_sort()`. This approach was chosen so that every step of the BFS process could be observed more clearly. According to Kumari & Singh (2024), topological sorting is important because it produces an ordering that follows all dependency relationships. A similar approach was also used by Liu & Lv (2023), where nodes with in-degree 0 are processed first.

CELL 7 - KAHN'S ALGORITHM (TOPOLOGICAL SORT BERBASIS BFS)

```
1 # Cell 7 - Kahn's Algorithm (Topological Sort berbasis BFS)
2 # Implementasi manual, tidak pakai nx.topological_sort()
3
4 def kahns_topological_sort(graph):
5     """
6     Kahn's Algorithm - BFS-based topological sort.
7     Returns:
8         topo_order : list node dalam urutan assembly yang valid
9         in_degree : dict in-degree akhir (semua 0 jika DAG valid)
10    """
11    in_degree = defaultdict(int)
12
13    # Hitung in-degree tiap node
14    for node in graph.nodes():
15        if node not in in_degree:
16            in_degree[node] = 0
17        for _ in graph.predecessors(node):
18            in_degree[node] += 1
19
20    # Queue: semua node dengan in-degree 0 (root nodes)
21    queue = deque(sorted([n for n, d in in_degree.items() if d == 0]))
22    topo_order = []
23
24    while queue:
25        node = queue.popleft()
26        topo_order.append(node)
27
28        # Kurangi in-degree semua successor
29        for successor in sorted(graph.successors(node)):
30            in_degree[successor] -= 1
31            if in_degree[successor] == 0:
32                queue.append(successor)
33
34    return topo_order, in_degree
35
36 topo_order, final_in_degree = kahns_topological_sort(G)
37
38 # Validasi: kalau semua node terproses + tidak ada cycle
39 is_valid = len(topo_order) == G.number_of_nodes()
40 print(f'Topological sort valid (no cycle) : {is_valid}')
41 print(f'Total node diproses : {len(topo_order)} / {G.number_of_nodes()}')
```

Gambar 5.14 Topological Sort Berbasis BFS

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The algorithm starts by calculating the in-degree of every node. All nodes with:

In-Degree=0

are placed into a queue as starting nodes. Each node is then processed one by one, added to the topological order, and its successors are updated until all nodes have been visited.

The result shows that all nodes were successfully processed without any cycle being detected:

```
Topological sort valid (no cycle) : True
Total node diproses : 103 / 103
```

Gambar 5.15 Output of Topological Sort Berbasis BFS

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

This confirms that the graph contains no cycles and a valid assembly sequence exists for all 103 components. The first positions in the sequence are occupied by the root nodes, while the final component is:

C103 – Full Car Systems Check & Sign-Off

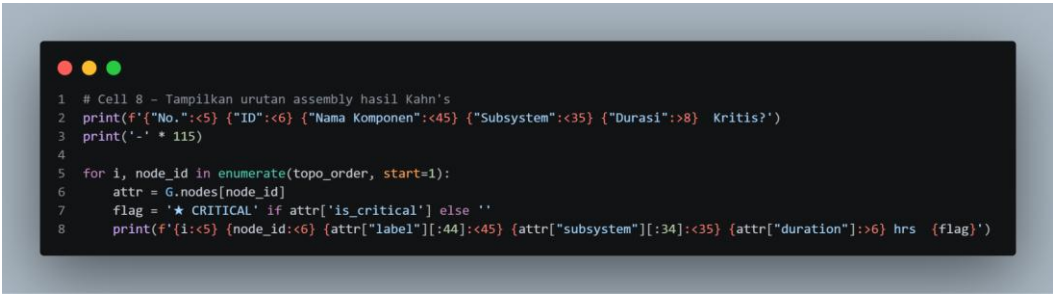
which represents the completion of the entire assembly process.

The algorithm runs with a time complexity of:

$$O(V+E)=O(103+216)=O(319)$$

CELL 8 — TOPOLOGICAL SORT RESULT

After running Kahn's Algorithm, a valid assembly sequence for all 103 Honda RA621H components was obtained. This sequence ensures that every component is processed only after all of its prerequisite components have been completed, preventing any dependency violations within the graph. According to Kumari & Singh (2024), topological sorting is used to generate an ordering of tasks while preserving all dependency relationships.

A screenshot of a Jupyter Notebook cell with a dark background and light-colored text. The code is in Python and uses the `enumerate` function to iterate over a topological order. It prints out details for each node, including its ID, name, subsystem, duration, and whether it is critical. The code is as follows:

```
1 # Cell 8 - Tampilkan urutan assembly hasil Kahn's
2 print(f'{"No.":<5} {"ID":<6} {"Nama Komponen":<45} {"Subsystem":<35} {"Durasi":>8} Kritis?}')
3 print('-' * 115)
4
5 for i, node_id in enumerate(topo_order, start=1):
6     attr = G.nodes[node_id]
7     flag = '★ CRITICAL' if attr['is_critical'] else ''
8     print(f'{i:<5} {node_id:<6} {attr["label"][:44]:<45} {attr["subsystem"][:34]:<35} {attr["duration"]:>6} hrs {flag}')
```

Gambar 5.16 Topological Sort Result

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The result shows that all nodes were successfully processed without detecting any cycle, confirming that the graph is a valid Directed Acyclic Graph (DAG). The first ten components are root nodes with an in-degree of zero, allowing them to be assembled in parallel at the beginning of the project. The remaining components appear according to the dependency structure defined in the graph.

No.	ID	Nama Komponen	Subsystem	Durasi	Kritis?
1	C001	V6 Cylinder Block (1.6L 90° Bank)	ICE – Internal Combustion Engine	48 hrs	★ CRITICAL
2	C018	Compressor Housing	Turbocharger System	18 hrs	★ CRITICAL
3	C025	MGU-H Rotor & Magnet Assembly	Hybrid – MGU-H	20 hrs	★ CRITICAL
4	C030	MGU-K Rotor & Magnet Assembly	Hybrid – MGU-K	18 hrs	★ CRITICAL
5	C035	Li-Ion Battery Cells (ES)	Energy Store (ES)	20 hrs	★ CRITICAL
6	C039	Dry-Sump Oil Tank	Lubrication System	6 hrs	
7	C043	Water Pump (Electric)	Cooling System	6 hrs	
8	C046	Fuel Tank (FIA Bladder, ~100kg capacity)	Fuel System	12 hrs	
9	C049	Engine Control Unit (ECU) – FIA Standard	Electronics & Control	8 hrs	★ CRITICAL
10	C052	Carbon Fibre Monocoque (Survival Cell)	Chassis & Monocoque	96 hrs	★ CRITICAL
11	C002	Crankshaft	ICE – Internal Combustion Engine	24 hrs	★ CRITICAL
12	C019	Compressor Wheel	Turbocharger System	14 hrs	★ CRITICAL
13	C026	MGU-H Stator Windings	Hybrid – MGU-H	16 hrs	★ CRITICAL
14	C031	MGU-K Stator Windings	Hybrid – MGU-K	14 hrs	★ CRITICAL
15	C036	Battery Management System (BMS)	Energy Store (ES)	12 hrs	★ CRITICAL
16	C040	Oil Pump & Scavenge Pumps	Lubrication System	8 hrs	★ CRITICAL
17	C044	Radiators (x2 side pods)	Cooling System	10 hrs	
18	C047	Fuel Pump & Pressure Regulator	Fuel System	6 hrs	★ CRITICAL
19	C050	Wiring Loom (PU section)	Electronics & Control	16 hrs	
20	C053	Roll Hoop (Halo + Main)	Chassis & Monocoque	24 hrs	★ CRITICAL
21	C054	Ballast Weights	Chassis & Monocoque	4 hrs	
22	C056	Front Upper Wishbone (x2)	Suspension – Front	12 hrs	
23	C057	Front Lower Wishbone (x2)	Suspension – Front	12 hrs	
...					
100	C100	Aero & Bodywork Installation	Final Assembly	20 hrs	★ CRITICAL
101	C101	Brakes, Wheels & Tyre Fitting	Final Assembly	8 hrs	★ CRITICAL
102	C102	Cockpit, Safety & Electronics Commissioning	Final Assembly	10 hrs	★ CRITICAL
103	C103	Full Car Systems Check & Sign-Off	Final Assembly	8 hrs	★ CRITICAL

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Gambar 5.17 Table Of Topological Sort Result

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Based on the generated sequence, C002 – Crankshaft can only be assembled after C001 – V6 Cylinder Block is completed, while C003 – Connecting Rods requires both C001 and C002 to be finished first. The final component in the sequence is C103 – Full Car Systems Check & Sign-Off, which marks the completion of the entire assembly process. The resulting topological order is then used as the foundation for the CPM calculations in the next stage.

CELL 9 — SUMMARY STATISTICS DAG + KAH'S ALGORITHM

```

1 # Cell 9 - Ringkasan akhir DAG + Kahn's
2 critical_nodes = [n for n in G.nodes() if G.nodes[n]['is_critical']]
3 avg_in = sum(dict(G.in_degree().values()).values()) / G.number_of_nodes()
4 avg_out = sum(dict(G.out_degree().values()).values()) / G.number_of_nodes()
5
6 # Cross-check dengan summary_stats.csv
7 assert G.number_of_nodes() == 103, f'Node count mismatch: {G.number_of_nodes()}'
8 assert G.number_of_edges() == 216, f'Edge count mismatch: {G.number_of_edges()}'
9 assert len(root_nodes) == 10, f'Root node mismatch: {len(root_nodes)}'
10 print('Assertions passed: node/edge/root count sesuai summary_stats.csv ✓')
11
12 print("\nRINGKASAN - Honda RA621H DAG + Kahn's Topological Sort")
13 print(f' Total nodes          : {G.number_of_nodes()}')
14 print(f' Total edges            : {G.number_of_edges()}')
15 print(f' Is acyclic (valid DAG)    : {nx.is_directed_acyclic_graph(G)}')
16 print(f' Root nodes                : {len(root_nodes)}')
17 print(f' Leaf nodes                : {len(leaf_nodes)}')
18 print(f' Critical path nodes       : {len(critical_nodes)}')
19 print(f' Avg in-degree             : {avg_in:.2f}')
20 print(f' Avg out-degree            : {avg_out:.2f}')
21 print(f' Kahn's sort valid         : {len(topo_order) == G.number_of_nodes()}')
22 print(f' Urutan pertama assembly   : {topo_order[0]} - {G.nodes[topo_order[0]]["label"]}')
23 print(f' Urutan terakhir assembly : {topo_order[-1]} - {G.nodes[topo_order[-1]]["label"]}')
24 print('\nDAG + Topological Sort selesai. Lanjut ke CPM.')

```

Gambar 5.18 Topological Sort Berbasis BFS

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

To verify the correctness of the graph and algorithm implementation, several summary statistics were generated from the dataset and graph structure.

```
Assertions passed: node/edge/root count sesuai summary_stats.csv ✓

RINGKASAN - Honda RA621H DAG + Kahn's Topological Sort
Total nodes           : 103
Total edges           : 216
Is acyclic (valid DAG) : True
Root nodes            : 10
Leaf nodes            : 8
Critical path nodes   : 52
Avg in-degree         : 2.10
Avg out-degree        : 2.10
Kahn's sort valid     : True
Urutan pertama assembly : C001 - V6 Cylinder Block (1.6L 90° Bank)
Urutan terakhir assembly : C103 - Full Car Systems Check & Sign-Off

DAG + Topological Sort selesai. Lanjut ke CPM.
```

Gambar 5.19 Summary

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The results confirm that the graph contains 103 components connected by 216 dependency relationships and satisfies the acyclic property required for a valid DAG. The graph contains 10 root nodes that can start immediately and 8 leaf nodes that do not act as prerequisites for other components.

The average in-degree and out-degree are both:

$$103 / 216 \approx 2.10$$

which indicates a balanced dependency structure. All statistics match the values stored in summary_stats.csv, confirming consistency between the original dataset and the generated graph.

D. STAGE 4 - CPM FORWARD PASS (CELL 10)

CELL 10

At this stage, the Forward Pass is performed to calculate the Earliest Start (ES) and Earliest Finish (EF) values for each component in the Honda RA621H assembly graph. The calculation is carried out using the `cpm_forward_pass(graph, topo_order)` function, which processes all nodes according to the topological order

generated in the previous stage.

According to Kumari & Singh (2024), CPM can be used to determine the project schedule by identifying the longest path in a dependency network. Therefore, the ES value of a component is obtained from the largest EF value among all of its predecessor nodes. For root nodes that do not have any predecessors, the ES value is set to 0. The EF value is then calculated using:

$$\mathbf{EF=ES+Duration}$$

Meanwhile, the ES value is determined by:

$$\mathbf{ES=\max(EF\ predeceessors)}$$

For example:

- C001 – V6 Cylinder Block (48 hrs) → ES = 0, EF = 48
- C002 – Crankshaft (24 hrs) → ES = 48, EF = 72
- C003 – Connecting Rods (12 hrs) → ES = max(48,72) = 72, EF = 84
- C006 – Cylinder Head (36 hrs) → ES = max(48,100) = 100, EF = 136

A screenshot of a Python script in a dark-themed code editor. The script defines a function `cpm_forward_pass` that takes a graph and a topological order as input. It iterates through the nodes in topological order, calculating the Earliest Start (ES) and Earliest Finish (EF) for each node. For root nodes (no predecessors), ES is set to 0. For other nodes, ES is the maximum EF of its predecessors. EF is then calculated as ES plus the node's duration. The script also calculates the total project duration and the EF of the final node.

```
1 def cpm_forward_pass(graph, topo_order):
2     """
3     Forward pass CPM - hitung ES dan EF tiap node.
4     ES[v] = max(EF[u]) untuk semua predecessor u
5     EF[v] = ES[v] + duration[v]
6     """
7     ES = {} # Earliest Start
8     EF = {} # Earliest Finish
9
10    for node in topo_order:
11        duration = graph.nodes[node]['duration']
12        predecessors = list(graph.predecessors(node))
13
14        if not predecessors:
15            ES[node] = 0 # root node mulai di t=0
16        else:
17            ES[node] = max(EF[p] for p in predecessors)
18
19        EF[node] = ES[node] + duration
20
21    return ES, EF
22
23
24 ES, EF = cpm_forward_pass(G, topo_order)
25
26 project_duration = max(EF.values())
27 print(f'Project Duration (Critical Path Length) : {project_duration} jam')
28 print(f'Earliest Finish node terakhir (C103) : {EF[topo_order[-1]]} jam')
```

Gambar 5.20 CPM Forward Pass

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The Output is :

```
Project Duration (Critical Path Length) : 358 jam
Earliest Finish node terakhir (C103)   : 358 jam
```

Gambar 5.21 Output of CPM Forward Pass

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The calculation results show that the total project duration is 358 hours, which is also the Earliest Finish value of the final component (C103). This means the assembly process cannot be completed in less than 358 hours unless the duration of one or more critical-path components is reduced.

E. STAGE 5 - CPM BACKWARD PASS (CELL 11)

CELL 11

After the Forward Pass is completed, the next step is the Backward Pass, which is used to calculate the Latest Finish (LF) and Latest Start (LS) values for each component. This process is performed using the `cpm_backward_pass(graph, topo_order, EF, project_duration)` function by traversing the graph in reverse topological order.

According to Kumari & Singh (2024), CPM identifies how much flexibility each activity has without affecting the overall project completion time. For leaf nodes, the LF value is set equal to the total project duration because these components must be completed before the project deadline. For all other nodes, LF is determined from the smallest LS value among its successor nodes.

The calculations use the following formulas:

$$LF = \min(LS \text{ successors})$$

$$LS = LF - \text{Duration}$$

```

1 def cpm_backward_pass(graph, topo_order, EF, project_duration):
2     """
3     Backward pass CPM - hitung LF dan LS tiap node.
4     LF[v] = min(LS[w]) untuk semua successor w
5     LS[v] = LF[v] - duration[v]
6     """
7     LF = {} # Latest Finish
8     LS = {} # Latest Start
9
10    for node in reversed(topo_order):
11        duration = graph.nodes[node]['duration']
12        successors = list(graph.successors(node))
13
14        if not successors:
15            LF[node] = project_duration # leaf node harus selesai tepat waktu
16        else:
17            LF[node] = min(LS[s] for s in successors)
18
19            LS[node] = LF[node] - duration
20
21    return LF, LS
22
23
24 LF, LS = cpm_backward_pass(G, topo_order, EF, project_duration)
25
26 print('Backward pass selesai.')
27 print(f'LS node pertama (harus 0) : {LS[topo_order[0]]}')
28 print(f'LF node terakhir : {LF[topo_order[-1]]} jam')

```

Gambar 5.21 CPM Backward Pass

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The Output is :

```

Backward pass selesai.
LS node pertama (harus 0) : 0
LF node terakhir : 358 jam

```

Gambar 5.22 Output of CPM Backward Pass

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The results show that the backward pass was completed successfully. One important validation is that the first component on the critical path, C001 – V6 Cylinder Block, has an LS value of 0. This means the component must start at the beginning of the project and cannot be delayed without affecting the final completion date.

F. STAGE 6 - FLOAT CALCULATION & CRITICAL PATH IDENTIFICATION

CELL 12

```

1 float_val = {} # Total Float per node
2 critical_path_nodes = []
3
4 for node in topo_order:
5     float_val[node] = LS[node] - ES[node] # ekuivalen: LF - EF
6     if float_val[node] == 0:
7         critical_path_nodes.append(node)
8
9 print(f'Jumlah node di Critical Path : {len(critical_path_nodes)}')
10 print(f'Project Duration : {project_duration} jam\n')
11
12 print(f'ID:<6> {"Nama Komponen":<45> {"ES":>6} {"EF":>6} {"LS":>6} {"LF":>6} {"Float":>6} Status}')
13 print('-' * 100)
14 for node in critical_path_nodes:
15     attr = G.nodes[node]
16     print(f'{node:<6} {attr["label"][:44]:<45} '
17           f'{ES[node]:>6} {EF[node]:>6} '
18           f'{LS[node]:>6} {LF[node]:>6} '
19           f'{float_val[node]:>6} ★ CRITICAL')

```

Gambar 5.23 Float Calculation & Critical Path Identification

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The Output is :

Jumlah node di Critical Path : 19							
Project Duration : 358 jam							
ID	Nama Komponen	ES	EF	LS	LF	Float	Status
C001	V6 Cylinder Block (1.6L 90° Bank)	0	48	0	48	0	★ CRITICAL
C002	Crankshaft	48	72	48	72	0	★ CRITICAL
C003	Connecting Rods (x6)	72	84	72	84	0	★ CRITICAL
C004	Pistons (x6)	84	100	84	100	0	★ CRITICAL
C006	Cylinder Head (x2 banks)	100	136	100	136	0	★ CRITICAL
C007	Intake Valves (x12)	136	144	136	144	0	★ CRITICAL
C008	Exhaust Valves (x12)	136	144	136	144	0	★ CRITICAL
C009	Camshafts (x4)	144	158	144	158	0	★ CRITICAL
C014	Exhaust Manifold (equal-length runners)	144	164	144	164	0	★ CRITICAL
C010	Valve Actuation – Pneumatic System	158	164	158	164	0	★ CRITICAL
C015	Engine Assembly & Bench Calibration	164	200	164	200	0	★ CRITICAL
C055	Mounting Points – Engine & Gearbox	200	208	200	208	0	★ CRITICAL
C077	Gearbox Casing (8-speed semi-auto)	208	256	208	256	0	★ CRITICAL
C078	Gear Stack & Selector Mechanism	256	288	256	288	0	★ CRITICAL
C079	Differential	288	302	288	302	0	★ CRITICAL
C098	Gearbox Mating to PU	302	314	302	314	0	★ CRITICAL
C099	Suspension Assembly – Full Car	314	330	314	330	0	★ CRITICAL
C100	Aero & Bodywork Installation	330	350	330	350	0	★ CRITICAL
C103	Full Car Systems Check & Sign-off	350	358	350	358	0	★ CRITICAL

Gambar 5.24 Critical Path in node

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

After the Forward Pass and Backward Pass are completed, the next step is calculating the Total Float for every component. According to Vazquez et al. (2023), float represents the amount of time an activity can be delayed without affecting the overall project completion date.

The float value is calculated using:

$$\text{Float} = \text{LS} - \text{ES} \text{ or equivalently: } \text{Float} = \text{LF} - \text{EF}$$

Components with Float = 0 are classified as Critical Path components because any delay in these activities will directly delay the entire project. Following the CPM calculation, the algorithm identifies 19 critical components that form the critical path:

C001 → C002 → C003 → C004 → C006 → C007 → C008 → C009 → C014
 → C010 → C015 → C055 → C077 → C078 → C079 → C098
 → C099 → C100 → C103

This path covers the main assembly flow from the ICE subsystem, through Chassis Mounting, Gearbox, and finally Final Assembly, spanning continuously from hour 0 to hour 358 without any scheduling slack.

CELL 13 — FULL CPM RESULTS TABLE



```

1 print(f'{"No.":<5} {"ID":<6} {"Nama Komponen":<42} {"Subsystem":<25} '
2       f'{"ES":>5} {"EF":>5} {"LS":>5} {"LF":>5} {"Float":>6} Kritis?}')
3 print('-' * 115)
4
5 for i, node in enumerate(topo_order, 1):
6     attr = G.nodes[node]
7     flag = '★ CP' if float_val[node] == 0 else ''
8     print(f'{i:<5} {node:<6} {attr["label"][:41]:<42} '
9           f'{attr["subsystem"].split("-")[-1].strip()[:24]:<25} '
10          f'{ES[node]:>5} {EF[node]:>5} '
11          f'{LS[node]:>5} {LF[node]:>5} '
12          f'{float_val[node]:>6} {flag}')

```

Gambar 5.25 CPM Result Table

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The Output is :

No.	ID	Nama Komponen	Subsystem	ES	EF	LS	LF	Float	Kritis?
1	C001	V6 Cylinder Block (1.6L 90° Bank)	Internal Combustion Engi	0	48	0	48	0	★ CP
2	C018	Compressor Housing	Turbocharger System	0	18	190	208	190	
3	C025	MGU-H Rotor & Magnet Assembly	MGU-H	0	20	194	214	194	
4	C030	MGU-K Rotor & Magnet Assembly	MGU-K	0	18	212	230	212	
5	C035	Li-Ion Battery Cells (ES)	Energy Store (ES)	0	20	220	240	220	
6	C039	Dry-Sump Oil Tank	Lubrication System	0	6	338	344	338	
7	C043	Water Pump (Electric)	Cooling System	0	6	336	342	336	
8	C046	Fuel Tank (FIA Bladder, ~100kg capacity)	Fuel System	0	12	336	348	336	
9	C049	Engine Control Unit (ECU) – FIA Standard	Electronics & Control	0	8	306	314	306	
10	C052	Carbon Fibre Monocoque (Survival Cell)	Chassis & Monocoque	0	96	104	200	104	
11	C002	Crankshaft	Internal Combustion Engi	48	72	48	72	0	★ CP
12	C019	Compressor Wheel	Turbocharger System	18	32	208	222	190	
13	C026	MGU-H Stator Windings	MGU-H	20	36	214	230	194	
14	C031	MGU-K Stator Windings	MGU-K	18	32	230	244	212	
15	C036	Battery Management System (BMS)	Energy Store (ES)	20	32	240	252	220	
16	C040	Oil Pump & Scavenge Pumps	Lubrication System	6	14	344	352	338	
17	C044	Radiators (x2 side pods)	Cooling System	6	16	342	352	336	
18	C047	Fuel Pump & Pressure Regulator	Fuel System	12	18	348	354	336	
19	C050	Wiring Loom (PU section)	Electronics & Control	8	24	314	330	306	
20	C053	Roll Hoop (Halo + Main)	Chassis & Monocoque	96	120	334	358	238	
21	C054	Ballast Weights	Chassis & Monocoque	96	100	354	358	258	
22	C056	Front Upper Wishbone (x2)	Front	96	108	282	294	186	
23	C057	Front Lower Wishbone (x2)	Front	96	108	282	294	186	
...									
100	C100	Aero & Bodywork Installation	Final Assembly	330	350	330	350	0	★ CP
101	C101	Brakes, Wheels & Tyre Fitting	Final Assembly	330	338	342	350	12	
102	C102	Cockpit, Safety & Electronics Commissioni	Final Assembly	330	340	340	350	10	
103	C103	Full Car Systems Check & Sign-Off	Final Assembly	350	358	350	358	0	★ CP

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Gambar 5.26 Output Of CPM Result Table

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Several components have large float values, indicating greater scheduling flexibility. For example, C052 – Carbon Fibre Monocoque has a float of 104 hours, while C039 – Dry-Sump Oil Tank has the highest float value at 338 hours. On the other hand, C101 and C102 have relatively small float values, making them close to becoming critical activities.

The results confirm that only 19 out of 103 components belong to the critical path, while the remaining components have varying levels of scheduling flexibility.

G. STAGE 7 - CPM VISUALIZATION (CELL 14)

CELL 14

To make the CPM results easier to understand, two visualizations are generated in this stage: a Float Distribution Histogram and a Critical Path Gantt Chart. These visualizations help illustrate the scheduling flexibility of each component and show how the critical path is formed throughout the Honda RA621H assembly process.

```

1 import numpy as np
2
3 df_cpm = df_comp.copy()
4 df_cpm['ES'] = df_cpm['Component ID'].map(ES)
5 df_cpm['EF'] = df_cpm['Component ID'].map(EF)
6 df_cpm['LS'] = df_cpm['Component ID'].map(LS)
7 df_cpm['LF'] = df_cpm['Component ID'].map(LF)
8 df_cpm['Float'] = df_cpm['Component ID'].map(float_val)
9 df_cpm['on_cp'] = df_cpm['Float'] == 0
10
11 # — Plot 1: Float histogram —————
12 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
13 fig.suptitle('CPM Analysis - Honda RA621H Assembly', fontsize=13, fontweight='bold')
14
15 axes[0].hist(df_cpm['Float'], bins=20, color='#E10600', edgecolor='black', alpha=0.85)
16 axes[0].axvline(0, color='#FFD700', linewidth=2, linestyle='--', label='Float = 0 (Critical)')
17 axes[0].set_title('Distribusi Total Float')
18 axes[0].set_xlabel('Total Float (jam)')
19 axes[0].set_ylabel('Jumlah Komponen')
20 axes[0].legend()
21
22 # — Plot 2: Gantt-style timeline critical path —————
23 cp_df = df_cpm[df_cpm['on_cp']].sort_values('ES').reset_index(drop=True)
24 colors_cp = plt.cm.Reds(np.linspace(0.4, 0.9, len(cp_df)))
25
26 axes[1].set_title(f'Critical Path Timeline ({len(cp_df)} komponen)')
27 for idx, row in cp_df.iterrows():
28     axes[1].barh(idx, row['Duration (hrs)'], left=row['ES'],
29                 color=colors_cp[idx], edgecolor='#333333', height=0.7)
30     axes[1].text(row['ES'] + row['Duration (hrs)'] / 2, idx,
31                 row['Component ID'], ha='center', va='center',
32                 fontsize=5.5, fontweight='bold', color='white')
33
34 axes[1].set_yticks(range(len(cp_df)))
35 axes[1].set_yticklabels([f'{r["Component ID"]}' for _, r in cp_df.iterrows()], fontsize=6)
36 axes[1].set_xlabel('Waktu (jam)')
37 axes[1].invert_yaxis()
38
39 plt.tight_layout()
40 plt.savefig('cpm_analysis.png', dpi=150, bbox_inches='tight')
41 plt.show()
42 print('Plot tersimpan: cpm_analysis.png')

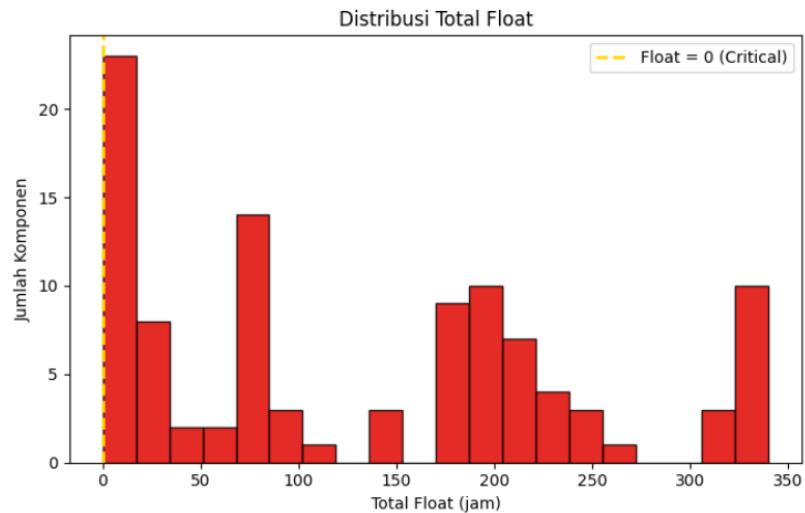
```

Gambar 5.27 CPM Visualization

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Float Distribution Histogram

The first graph displays the distribution of Float values across all 103 components. A dashed gold line is added at Float = 0 to clearly separate critical components from non-critical ones.



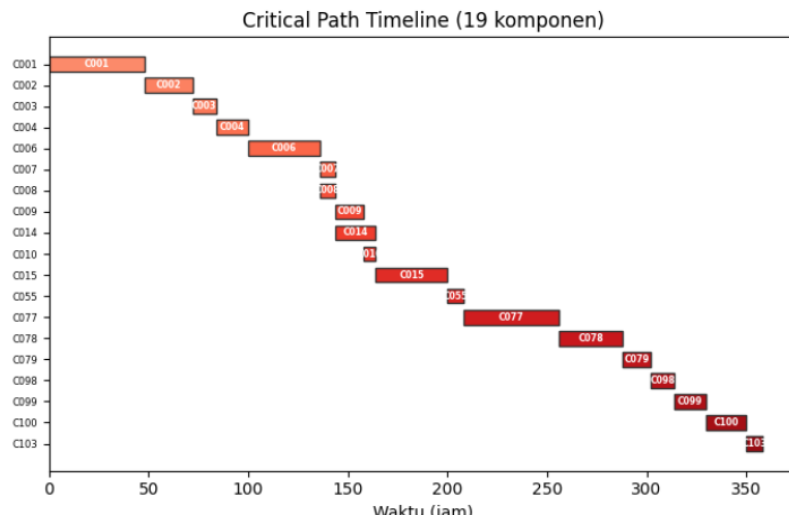
Gambar 5.28 Histogram Of CPM Float Distribution

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The histogram shows that many components have float values close to zero, indicating that they are critical or near-critical activities. At the same time, several components have large float values, meaning they can be delayed without affecting the overall project duration. This pattern reflects the structure of the Honda RA621H assembly process, where a critical assembly chain exists alongside several parallel activities with greater scheduling flexibility.

Critical Path Gantt Chart

The second graph visualizes the 19 critical-path components using a Gantt chart. Each bar represents a component and spans from its Earliest Start (ES) to Earliest Finish (EF) time.



Gambar 5.29 Chart Of Critical Path Gantt

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

The chart clearly shows a continuous sequence from C001 at hour 0 to C103 at hour 358. Since all critical-path components have Float = 0, there are no gaps or scheduling flexibility along this path. Any delay in one of these activities will directly delay the completion of the entire project.

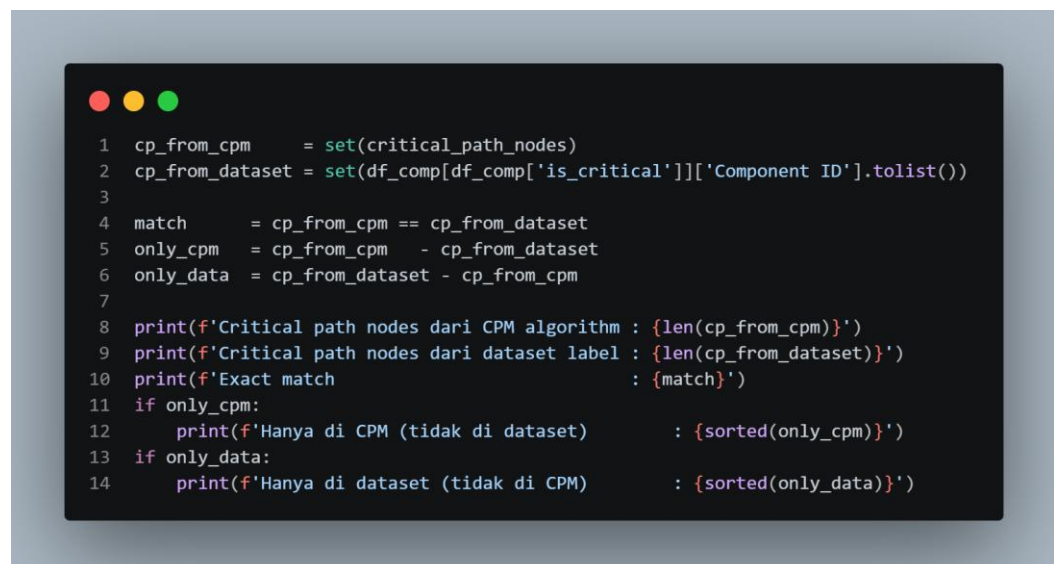
Both visualizations are saved as:

cpm_analysis.png

and provide a visual summary of the CPM analysis results.

H. STAGE 8 - VALIDATION: CPM RESULTS VS DATASET LABELS

CELL 15

A screenshot of a Jupyter Notebook cell with a dark background and light-colored text. The code is written in Python and compares critical path nodes from a CPM algorithm with dataset labels. It uses sets to find matches and differences. The output is printed as formatted strings. The code is as follows:

```
1 cp_from_cpm = set(critical_path_nodes)
2 cp_from_dataset = set(df_comp[df_comp['is_critical']]['Component ID'].tolist())
3
4 match = cp_from_cpm == cp_from_dataset
5 only_cpm = cp_from_cpm - cp_from_dataset
6 only_data = cp_from_dataset - cp_from_cpm
7
8 print(f'Critical path nodes dari CPM algorithm : {len(cp_from_cpm)}')
9 print(f'Critical path nodes dari dataset label : {len(cp_from_dataset)}')
10 print(f'Exact match : {match}')
11 if only_cpm:
12     print(f'Hanya di CPM (tidak di dataset) : {sorted(only_cpm)}')
13 if only_data:
14     print(f'Hanya di dataset (tidak di CPM) : {sorted(only_data)}')
```

Gambar 5.30 Code Of Validation for CPM

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

After all CPM calculations are completed, the results are compared with the critical labels provided in the original dataset. The CPM algorithm identifies 19 mathematically critical components, which are components with:

Float=0

According to Vazquez et al. (2023), activities with Float = 0 belong to the critical path because any delay in these activities will directly delay the project completion time.

The comparison shows that the CPM results and the dataset labels are not exactly the same (Exact Match = False).

```

Critical path nodes dari CPM algorithm : 19
Critical path nodes dari dataset label : 52
Exact match : False
Hanya di CPM (tidak di dataset) : ['C007', 'C008', 'C010']
Hanya di dataset (tidak di CPM) : ['C012', 'C016', 'C017', 'C018', 'C019', 'C020', 'C024', 'C025', 'C026', 'C027', 'C029', 'C030', 'C031', 'C032', 'C033']

```

Gambar 5.31 Output

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Several components such as **C007 (Intake Valves)**, **C008 (Exhaust Valves)**, and **C010 (Valve Actuation – Pneumatic System)** appear in the CPM critical path but are not marked as critical in the original dataset. Since these components have Float = 0, the algorithm automatically classifies them as critical.

On the other hand, there are also components that are labeled as critical in the dataset but do not appear on the CPM critical path. Examples include **C052 (Carbon Fibre Monocoque)**, **C097 (PU Installation into Chassis)**, **C101 (Brakes & Wheels Fitting)**, and **C018 (Compressor Housing)**. Although these components have non-zero float values, they may still be considered important from an engineering perspective because of their mechanical importance, safety requirements, or operational risks.

This comparison highlights the difference between **mathematically critical components** identified by CPM and **operationally critical components** identified through engineering judgment. The CPM results are useful for scheduling and resource planning, while the dataset labels provide additional insight for risk management and quality control.

I. STAGE 9 - FINAL SUMMARY

CELL 16

```

1 print('=' * 65)
2 print('Summary - Honda RA621H | DAG + Kahn\'s + CPM')
3 print('=' * 65)
4 print(f' Total komponen (nodes) : {G.number_of_nodes()}')
5 print(f' Total dependency (edges) : {G.number_of_edges()}')
6 print(f' Root nodes : {len(root_nodes)}')
7 print(f' Leaf nodes : {len(leaf_nodes)}')
8 print(f' Is valid DAG (acyclic) : {nx.is_directed_acyclic_graph(G)}')
9 print(f' Kahn\'s sort valid : {len(topo_order) == G.number_of_nodes()}')
10 print()
11 print(f' Project Duration (CPM) : {project_duration} jam')
12 print(f' Critical path nodes : {len(critical_path_nodes)}')
13 print(f' Non-critical (ada slack) : {G.number_of_nodes() - len(critical_path_nodes)}')
14 print(f' Max float (paling fleksibel) : {int(max(float_val.values()))} jam')
15 print()
16 print(f' Critical path mulai dari : {critical_path_nodes[0]} - {G.nodes[critical_path_nodes[0]]["label"]}')
17 print(f' Critical path berakhir di : {critical_path_nodes[-1]} - {G.nodes[critical_path_nodes[-1]]["label"]}')

```

Gambar 5.32 Code for Final Summary

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

This final stage summarizes the overall results of the Honda RA621H assembly scheduling analysis. The graph consists of 103 components (nodes) and 216 dependency relationships (edges), forming a valid Directed Acyclic Graph (DAG) with no cycles detected. The assembly process begins from 10 root nodes and ends at 8 leaf nodes, with the final assembly stage concluding at C103 – Full Car Systems Check & Sign-Off.

```

=====
Summary - Honda RA621H | DAG + Kahn's + CPM
=====
Total komponen (nodes)      : 103
Total dependency (edges)    : 216
Root nodes                   : 10
Leaf nodes                   : 8
Is valid DAG (acyclic)      : True
Kahn's sort valid           : True

Project Duration (CPM)      : 358 jam
Critical path nodes         : 19
Non-critical (ada slack)    : 84
Max float (paling fleksibel): 340 jam

Critical path mulai dari    : C001 - V6 Cylinder Block (1.6L 90° Bank)
Critical path berakhir di   : C103 - Full Car Systems Check & Sign-Off

```

Gambar 5.33 SUMMARY

Source : Notebook Project Honda_RA621H_DAG_Kahn_CPM.ipynb (Personal documentation)

Using Kahn's Algorithm, a valid topological order was successfully generated for all 103 components, confirming that every dependency relationship is satisfied. This result is consistent with the role of topological sorting described by Kumari & Singh (2024), where tasks are arranged in an order that respects all prerequisite constraints.

The CPM analysis shows that the minimum time required to complete the entire assembly process is 358 hours. From the 103 components, 19 components were identified as part of the critical path, meaning they have:

Float=0

and therefore cannot be delayed without affecting the overall project duration. The remaining 84 components have positive float values, providing varying levels of scheduling flexibility.

The critical path starts from C001 – V6 Cylinder Block (1.6L 90° Bank) and ends at C103 – Full Car Systems Check & Sign-Off, forming the main backbone of the assembly process. As explained by Vazquez et al. (2023), activities on the

critical path directly determine the project completion time, making them the most important tasks to monitor during execution.

Overall, the results demonstrate that DAG representation, Kahn's Topological Sort, and CPM can be effectively applied to model and analyze a complex real-world assembly process such as the Honda RA621H Formula 1 power unit.

CHAPTER 6

ATTACHMENT

6.1. Project Files and Resources

The following project files can be accessed through the links below:

Python Notebook (.ipynb):

[PROJECT](#)

Web Visualization Files (index.html, css/, js/):

[Project Final.html](#)

Complete Project Folder:

[PROJECT](#)

[GITHUB](#)

Project Final Report :

[LAPORAN AKHIR PEMDAS DEGICHI](#)

dalam bentuk word atau pdf tersedia di drive berikut : [PROJECT](#)

Personal Documentation :

[others](#)

At the time of writing, all source code, datasets, visualizations, and supporting files are stored in the project repository and can be accessed through the links provided above for reference, verification, and further development.

REFERENCES

- Kumari, M., & Singh, G.P. (2024).** The Application of Graph Theory to Solving CPM and PERT Problems in Java. *International Research Journal of Management Science & Technology (IRJMST)*, 15(8), 20–28. ISSN 2250-1959.
- Liu, J., & Lv, Y. (2023).** A Multi-Object Genetic Algorithm for the Assembly Line Balance Optimization in Garment Flexible Job Shop Scheduling. *Intelligent Automation & Soft Computing (IASC)*, 37(2), 2421–2439. <https://doi.org/10.32604/iasc.2023.040262>
- Vazquez, A., Pozzana, I., Kalogridis, G., & Ellinas, C. (2023).** Activity networks determine project performance. *Scientific Reports*, 13, 509. <https://doi.org/10.1038/s41598-022-27180-0>